MOBILE PROGRAMMING II

Unit Pengembang Akademi
PRODI SISTEM INFORMASI



FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS BINA SARANA INFORMATIKA

2025

KATA PENGANTAR

Puja dan puji syukur selalu kami panjatkan kehadirat Allah Swt yang telah memberikan semua nikmatnya sehingga penulis berhasil menyelesaikan buku yang berjudul Mobile Programming ini tanpa adanya kendala yang berarti. Tujuan dari penyusunan modul ini adalah untuk memudahkan para mahasiswa Sistem Informasi Fakultas Teknik dan Informatika Universitas Bina Sarana Informatika dalam mengenl dan memahami mobile programming dan praktik pembuatan API. Modul ini disusun dalam tahapan agar pemula dapat lebih mudah mempelajarai mobile programming dan mengembangkan aplikasi mobile yang dikhususkan kepada sistem operasi android.

Keberhasilan penyusunan modul ini tentunya bukan atas usaha penulis saja namun ada banyak pihak yang turut membantu dan memberikan dukungan untuk suksesnya penulisan modul ini. Untuk itu, penulis mengucapkan terima kasih yang sebesar-besarnya kepada semua pihak yang telah memberikan dukungan baik secara moril ataupun material sehingga buku ini berhasil disusun.

Modul yang ada ini tentu tidak luput dari kekurangan. Selalu ada celah untuk perbaikan. Kritik, saran serta masukkan dari pembaca sangat kami harapkan untuk penyempurnaan kedepannya.

Pontianak, Januari 2024

Tim Penulis

DAFTAR ISI

KATA PENGANTAR 1
DAFTAR ISI 2
PERTEMUAN 1 4
Penjelasan Tugas Project 4
PERTEMUAN 2 5
Membuat Halaman Data Poli 5
Membuat Detail Poli 7
Menampilkan Detail Poli saat Data Poli diklik 9
TUGAS 12
PERTEMUAN 3 13
Memisahkan List Poli menjadi Poli Item 13
Membuat Form Tambah Poli 14
TUGAS 16
PERTEMUAN 4 17
Pemisahan Widget Kedalam Fungsi-Fungsi 17
Membuat Fungsi Tombol Simpan dan Menampilkan Data pada Detail Poli 18
PERTEMUAN 5 20
Membuat Form Ubah Poli 20
Menampilkan Pesan Konfirmasi Hapus Data 22
TUGAS 24
PERTEMUAN 6 25
Membuat Halaman Beranda 25
Membuat Halaman Login 25
Membuat Sidebar Menu (Drawer) 26
TUGAS 30
PERTEMUAN 7 31
Apa itu API 31
Arsitektur API 31
Install Postman 32
PERTEMUAN 8 34

UJIAN TENGAH SEMESTER 34
PERTEMUAN 9 35
Membuat API Klinik dummy dengan mockapi.io 35
Create Poli dengan method POST 39
List Poli dengan method GET 39
Show dengan method GET 39
Update Poli dengan method PUT 39
Delete Poli dengan method DELETE 40
TUGAS 40
PERTEMUAN 10 41
Menambahkan Depedencies 41
Membuat Class Untuk Menyimpan Token 41
Mengarahkan Halaman Awal ke Login 42
PERTEMUAN 11 43
Persiapan Koneksi ke API 43
Membuat Service Login 44
Membuat Service Poli 45
TUGAS 46
PERTEMUAN 12 47
Mengambil List Poli dari API 47
Menambah Data Poli ke API 49
TUGAS 50
PERTEMUAN 13 51
Mengambil Detail Poli dari API 51
Menghapus Data Poli dari API 53
Mengubah Data Poli dari API 56
TUGAS 57
PERTEMUAN 14 58
Presentasi Projek 58
PERTEMUAN 15 58
Presentasi Projek 58
PERTEMUAN 16 58
Ujian Akhir Semester 58

PERTEMUAN 1

Penjelasan Tugas Project

Tugas project diadakan untuk memperoleh nilai UTS dan UAS, dengan kata lain tugas projek ini sebagai pengganti UTS dan UAS. Tugas ini dikerjakan secara kelompok dengan maksimal 1 kelompok sebanyak 5 mahasiswa atau lebih disesuaikan dengan jumlah mahasiswa dalam satu kelas.

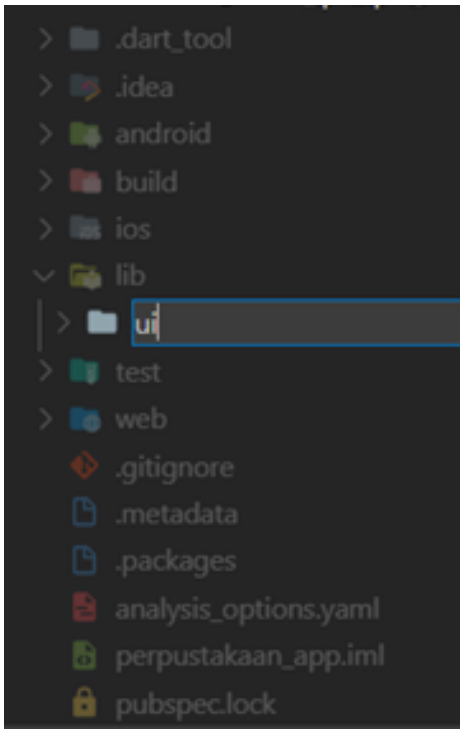
Bentuk Tugas Projek :

1. Projek merupakan program aplikasi (CRUD) menggunakan flutter dengan minimal 3 tabel.
2. Projek dipresentasikan pada pertemuan 14 dan 15

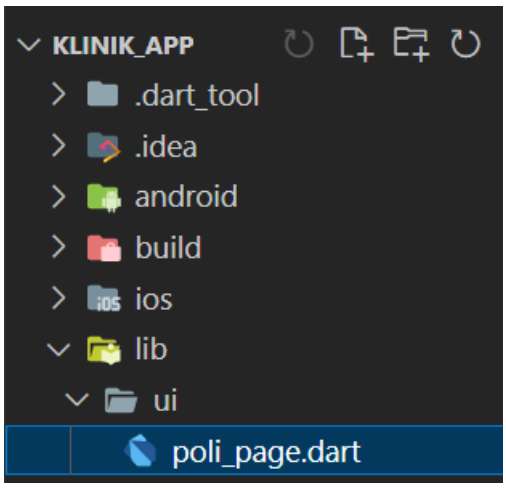
PERTEMUAN 2

Membuat Halaman Data Poli

Buat sebuah projek dengan nama **klinik_app**, selanjutnya kita akan belajar membuat form pada flutter, agar lebih rapi untuk tampilan halaman akan kita kelompokkan dalam sebuah folder tersendiri, dalam hal ini kita membuat folder dengan nama **ui** didalam folder **lib.**



Kemudian didalam folder **ui** tersebut kita buat sebuah file dengan nama poli_page.dart



Kemudian Ketikkan kode berikut

```dart
import 'package:flutter/material.dart';

class PoliPage extends StatefulWidget {
  const PoliPage({super.key});

  @override
  State<PoliPage> createState() => _PoliPageState();
}

class _PoliPageState extends State<PoliPage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Data Poli")),
      body: ListView(
        children: const [
          Card(
            child: ListTile(
              title: const Text("Poli Anak"),
            ),
          ),
          Card(
            child: ListTile(
              title: const Text("Poli Kandungan"),
            ),
          ),
          Card(
            child: ListTile(
              title: const Text("Poli Gigi"),
            ),
          ),
          Card(
            child: ListTile(
              title: const Text("Poli THT"),
            ),
          )
        ],
      ),
    );
  }
}
```

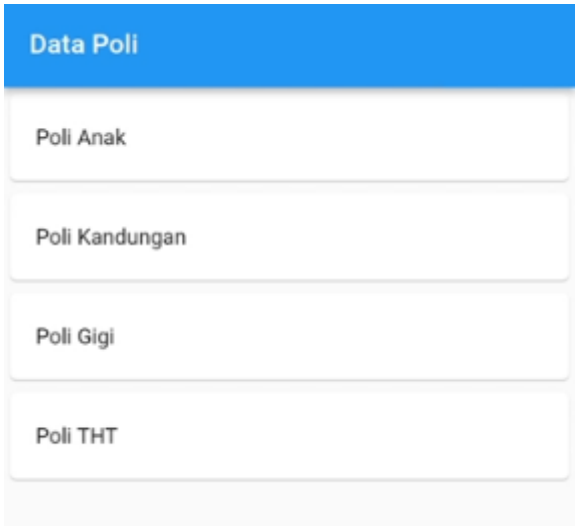
Ubah pada **main.dart** dengan memanggil class **PoliPage**

```dart
import 'package:flutter/material.dart';
import './ui/poli_page.dart';

void main() => runApp(MyApp());

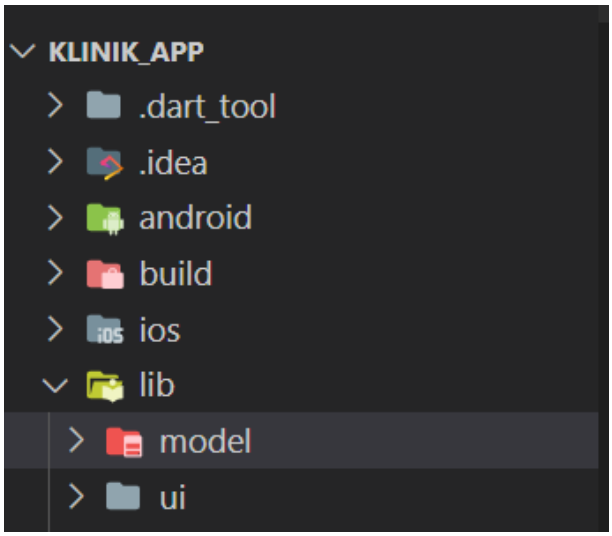
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Klinik APP',
      debugShowCheckedModeBanner: false,
      home: PoliPage(),
    );
  }
}
```

Sehingga hasilnya akan menjadi

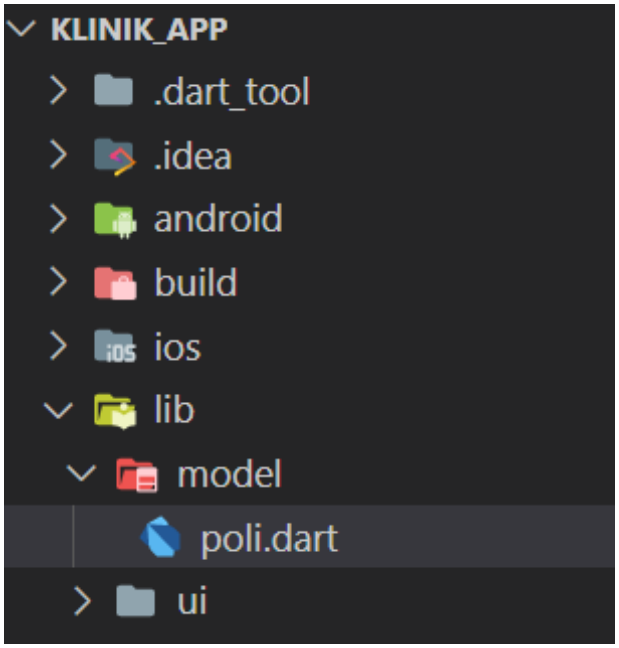


Membuat Detail Poli

Di dalam folder **lib** buat sebuah folder dengan nama **model**



Kemudian di dalam folder **model** tersebut, buat sebuah file dengan nama **poli.dart**



Di dalam file **poli.dart** tersebut ketikkan kode berikut

```dart
class Poli {
  String? id;
  String namaPoli;

  Poli({this.id, required this.namaPoli});
}
```

Buat sebuah file dengan nama **poli_detail.dart** di dalam folder **ui**, kemudian ketikkan kode berikut

```dart
import 'package:flutter/material.dart';
import '../model/poli.dart';

class PoliDetail extends StatefulWidget {
  final Poli poli;

  const PoliDetail({super.key, required this.poli});

  @override
  State<PoliDetail> createState() => _PoliDetailState();
}

class _PoliDetailState extends State<PoliDetail> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Detail Poli")),
      body: Column(
        children: [
          SizedBox(height: 20),
          Text(
            "Nama Poli : ${widget.poli.namaPoli}",
            style: TextStyle(fontSize: 20),
          ),
```

```
25.           SizedBox(height: 20),
26.           Row(
27.             mainAxisAlignment: MainAxisAlignment.spaceEvenly,
28.             children: [
29.               ElevatedButton(
30.                   onPressed: () {},
31.                   style:
32.                       ElevatedButton.styleFrom(backgroundColor: Colors.green),
33.                   child: const Text("Ubah")),
34.               ElevatedButton(
35.                   onPressed: () {},
36.                   style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
37.                   child: const Text("Hapus")),
38.             ],
39.           )
40.         ],
41.       ),
42.     );
43.   }
44. }
```

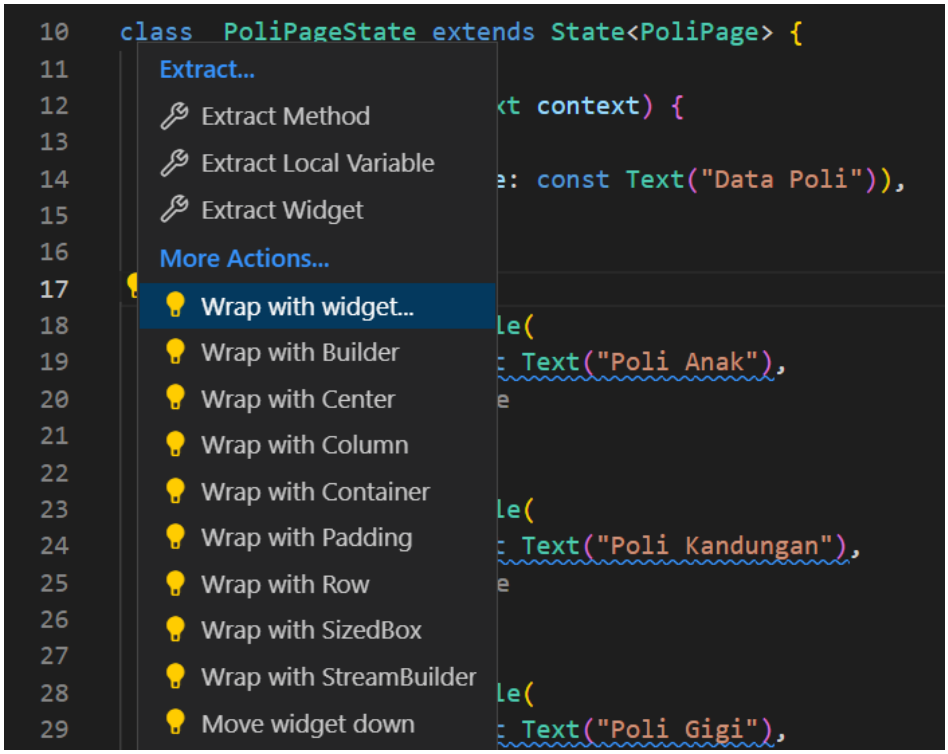
Menampilkan Detail Poli saat Data Poli diklik

Pada bagian ini kita akan menambahkan sebuah fungsi dimana saat salah satu Data Poli di klik, maka akan membuka halaman Detail Poli yang telah dibuat sebelumnya

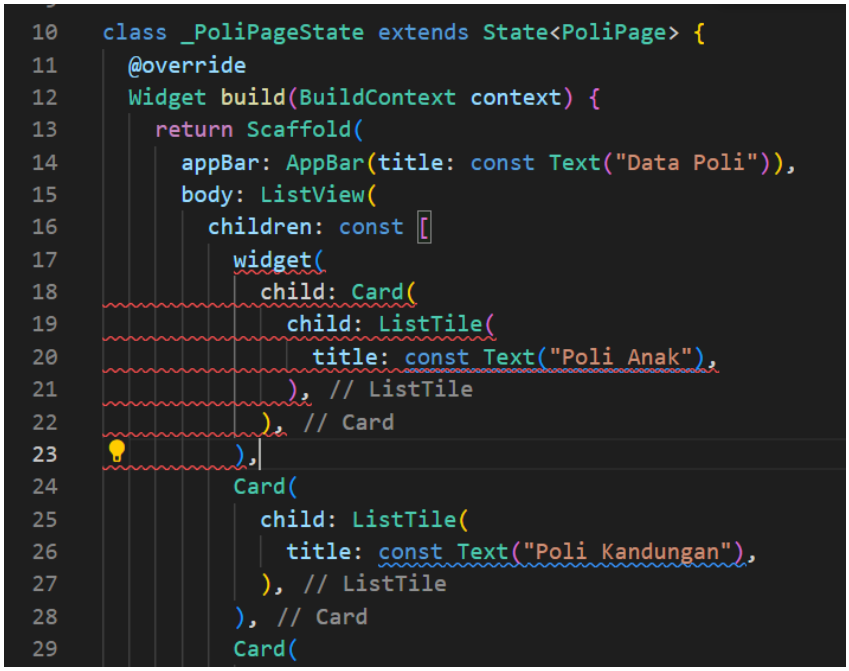
Kita akan memodifikasi Class PoliPage pada file **poli_page.dart**. Untuk menambahkan widget diatas widget lain yang telah dibuat dapat dilakukan dengan cara, arahkan kursor pada widget, misalnya dalam hal ini adalah widget **Card**

```
10  class _PoliPageState extends State<PoliPage> {
11    @override
12    Widget build(BuildContext context) {
13      return Scaffold(
14        appBar: AppBar(title: const Text("Data Poli")),
15        body: ListView(
16          children: const [
17            Card(
18              child: ListTile(
19                title: const Text("Poli Anak"),
20              ), // ListTile
21            ), // Card
22            Card(
23              child: ListTile(
24                title: const Text("Poli Kandungan"),
25              ), // ListTile
26            ), // Card
27            Card(
28              child: ListTile(
29                title: const Text("Poli Gigi"),
```

Pada bagian kiri akan muncul logo lampu, kemudian klik lampu tersebut dan pilih widget yang ingin ditambahkan atau dalam hal ini kita akan memilih **Warp with widget..**



Kemudian akan menjadi



Setelah itu ubah **widget** menjadi GestureDetector dan kita juga mengimport **poli_detail.dart** serta menambahkan onTap yang kemudian akan membuka halaman Detail Poli, sehingga kode untuk Class PoliPage menjadi

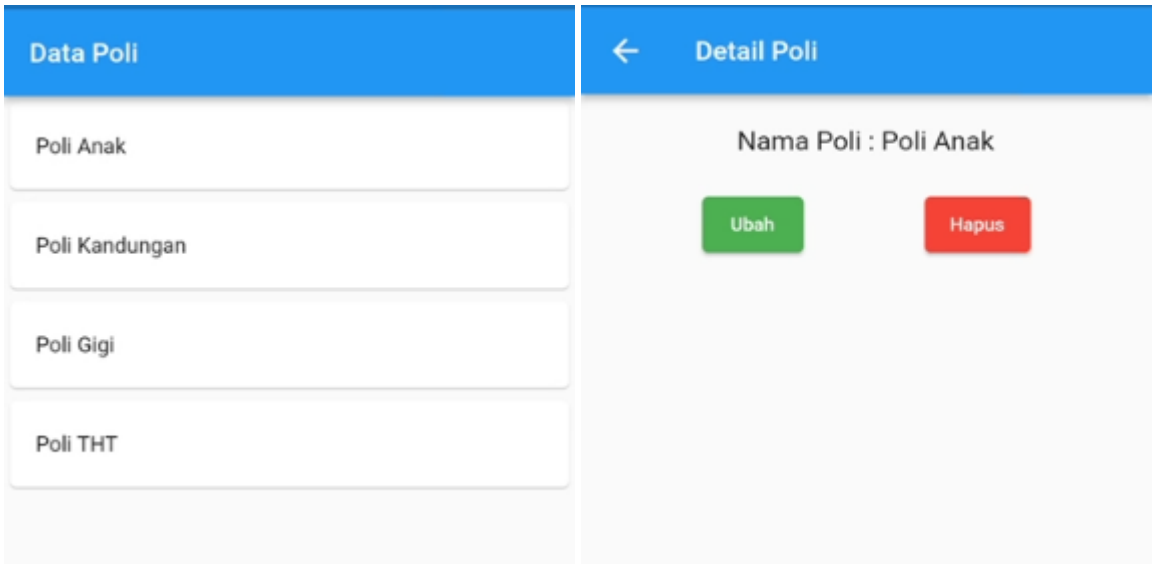
```dart
import 'package:flutter/material.dart';
import '../model/poli.dart';
import 'poli_detail.dart';

class PoliPage extends StatefulWidget {
  const PoliPage({super.key});

  @override
  State<PoliPage> createState() => _PoliPageState();
}

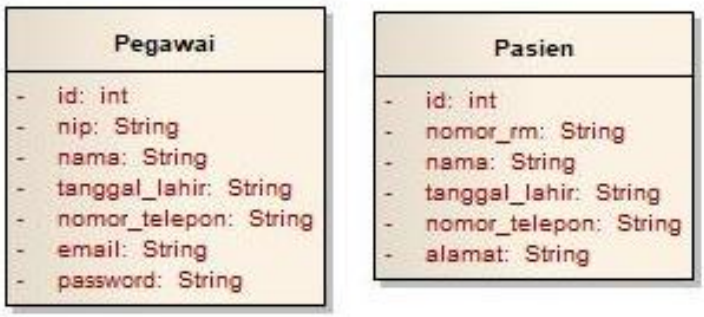
class _PoliPageState extends State<PoliPage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Data Poli")),
      body: ListView(
        children: [
          GestureDetector(
            child: Card(
              child: ListTile(
                title: const Text("Poli Anak"),
              ),
            ),
            onTap: () {
              Poli poliAnak = new Poli(namaPoli: "Poli Anak");
              Navigator.push(
                  context,
                  MaterialPageRoute(
                      builder: (context) => PoliDetail(poli: poliAnak)));
            },
          ),
          Card(
            child: ListTile(
              title: const Text("Poli Kandungan"),
            ),
          ),
          Card(
            child: ListTile(
              title: const Text("Poli Gigi"),
            ),
          ),
          Card(
            child: ListTile(
              title: const Text("Poli THT"),
            ),
          )
        ],
      ),
    );
  }
}
```

Jika dijalankan, maka pada list “Poli Anak” dapat kita klik dan akan menuju ke halaman Detail Poli



TUGAS

Buat sebagaimana halnya pertemuan 5 (Membuat Halaman menampilkan list data dan Membuat Halaman Detail Data) dengan skema seperti gambar dibawah



PERTEMUAN 3

Memisahkan List Poli menjadi Poli Item

Jika dilihat pada list lainnya maka fungsi pindah ke halaman Detail Poli tidak akan berfungsi, karena kita harus membuat hal yang sama seperti pada list "Poli Anak". Namun cara seperti ini tidak akan efektif, karena akan banyak kode dan menjadi tidak rapi. Solusinya adalah kita akan membuat class PoliItem yang akan menampilkan data Poli.

Buat file dengan nama **poli_item.dart** pada folder **ui** dan ketikkan kode berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import 'poli_detail.dart';
4.
5.  class PoliItem extends StatelessWidget {
6.    final Poli poli;
7.
8.    const PoliItem({super.key, required this.poli});
9.
10.   @override
11.   Widget build(BuildContext context) {
12.     return GestureDetector(
13.       child: Card(
14.         child: ListTile(
15.           title: Text("${poli.namaPoli}"),
16.         ),
17.       ),
18.       onTap: () {
19.         Navigator.push(context,
20.             MaterialPageRoute(builder: (context) => PoliDetail(poli: poli)));

21.       },
22.     );
23.   }
24. }
```

GestureDetector adalah widget yang digunakan untuk mendeteksi gesture pada widget seperti gesture ontap, doubletab dan lain-lain.

Kemudian buka file **poli_page.dart** dan ubah menjadi

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import 'poli_detail.dart';
4.  import 'poli_item.dart';
5.
6.  class PoliPage extends StatefulWidget {
7.    const PoliPage({super.key});
8.
9.    @override
10.   State<PoliPage> createState() => _PoliPageState();
11. }
12.
13. class _PoliPageState extends State<PoliPage> {
14.   @override
15.   Widget build(BuildContext context) {
16.     return Scaffold(
17.       appBar: AppBar(title: const Text("Data Poli")),
18.       body: ListView(
```

```
        children: [
          PoliItem(poli: Poli(namaPoli: "Poli Anak")),
          PoliItem(poli: Poli(namaPoli: "Poli Kandungan")),
          PoliItem(poli: Poli(namaPoli: "Poli Gigi")),
          PoliItem(poli: Poli(namaPoli: "Poli THT")),
        ],
      ),
    );
  }
}
```

Membuat Form Tambah Poli

Selanjutnya kita akan belajar membuat form pada flutter, buat sebuah file dengan nama **poli_form.dart** didalam folder **ui**, kemudian ketikkan kode berikut.

```
import 'package:flutter/material.dart';

class PoliForm extends StatefulWidget {
  const PoliForm({Key? key}) : super(key: key);
  _PoliFormState createState() => _PoliFormState();
}

class _PoliFormState extends State<PoliForm> {
  final _formKey = GlobalKey<FormState>();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Tambah Poli")),
      body: SingleChildScrollView(
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextField(
                  decoration: const InputDecoration(labelText: "Nama Poli")),
              SizedBox(height: 20),
              ElevatedButton(onPressed: () {}, child: const Text("Simpan"))
            ],
          ),
        ),
      ),
    );
  }
}
```

Selanjutnya Form Poli diatas akan dibuka melalui halaman Data Poli. Buka kembali **poli_page.dart** kemudian pada bagian **AppBar** tambahkan kode berikut

```
AppBar(
        title: const Text("Data Poli"),
        actions: [
          GestureDetector(
            child: const Icon(Icons.add),
            onTap: () {
              Navigator.push(
                  context, MaterialPageRoute(builder: (context) => PoliForm()));
            },
          )
        ],
      )
```

Sehingga secara keseluruhan kode tersebut menjadi

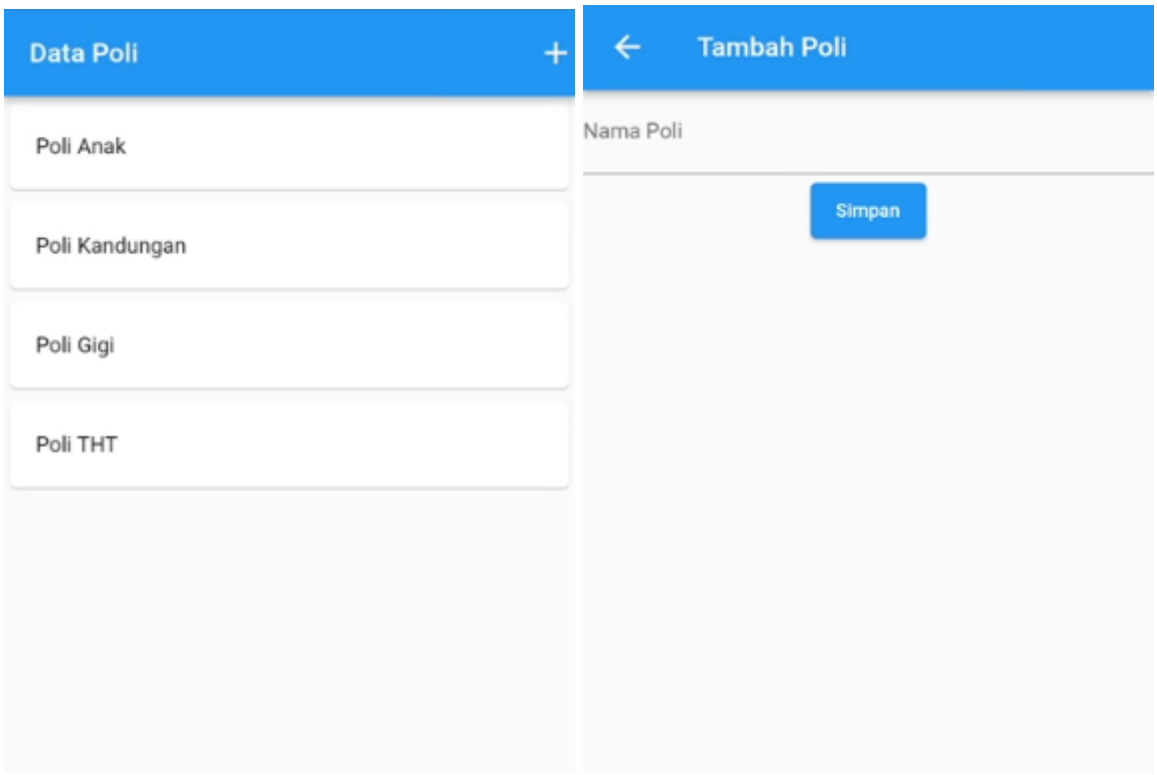
```dart
import 'package:flutter/material.dart';
import '../model/poli.dart';
import 'poli_detail.dart';
import 'poli_item.dart';
import 'poli_form.dart';

class PoliPage extends StatefulWidget {
  const PoliPage({super.key});

  @override
  State<PoliPage> createState() => _PoliPageState();
}

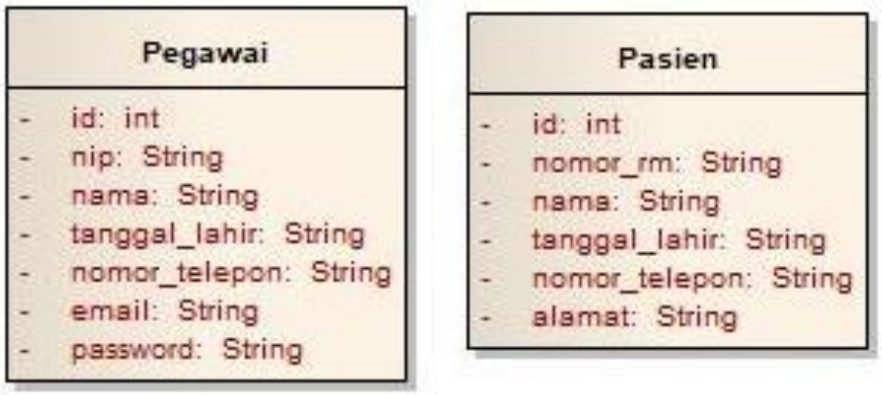
class _PoliPageState extends State<PoliPage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Data Poli"),
        actions: [
          GestureDetector(
            child: const Icon(Icons.add),
            onTap: () {
              Navigator.push(
                  context, MaterialPageRoute(builder: (context) => PoliForm()));
            },
          )
        ],
      ),
      body: ListView(
        children: [
          PoliItem(poli: Poli(namaPoli: "Poli Anak")),
          PoliItem(poli: Poli(namaPoli: "Poli Kandungan")),
          PoliItem(poli: Poli(namaPoli: "Poli Gigi")),
          PoliItem(poli: Poli(namaPoli: "Poli THT")),
        ],
      ),
    );
  }
}
```

Hasilnya akan muncul icon + pada bagian kanan AppBar, jika diklik akan membuka PoliForm



TUGAS

Buat sebagaimana halnya pertemuan 4 (Memisahkan class item dan Membuat Form tambah) dengan skema seperti gambar dibawah



PERTEMUAN 4

Pemisahan Widget Kedalam Fungsi-Fungsi

Agar kode mudah dibaca dan dikembangkan, akan lebih baik jika widget-widget yang digunakan dipisahkan kedalam method/function tertentu, misalnya pada **poli_form.dart** terdapat widget seperti **TextField dan ElevatedButton**, pada widget tersebut akan kita pisahkan kedalam method tersendiri didalam class, sehingga menjadi seperti berikut

```
1.  import 'package:flutter/material.dart';
2.
3.  class PoliForm extends StatefulWidget {
4.    const PoliForm({Key? key}) : super(key: key);
5.    _PoliFormState createState() => _PoliFormState();
6.  }
7.
8.  class _PoliFormState extends State<PoliForm> {
9.    final _formKey = GlobalKey<FormState>();
10.
11.   @override
12.   Widget build(BuildContext context) {
13.     return Scaffold(
14.       appBar: AppBar(title: const Text("Tambah Poli")),
15.       body: SingleChildScrollView(
16.         child: Form(
17.           key: _formKey,
18.           child: Column(
19.             children: [_fieldNamaPoli(), SizedBox(height: 20), _tombolSimpan()],
20.           ),
21.         ),
22.       ),
23.     );
24.   }
25.
26.   _fieldNamaPoli() {
27.     return TextField(
28.       decoration: const InputDecoration(labelText: "Nama Poli"),
29.     );
30.   }
31.
32.   _tombolSimpan() {
33.     return ElevatedButton(
34.         onPressed: () {},
35.         child: const Text("Simpan"));
36.   }
37. }
```

Membuat Fungsi Tombol Simpan dan Menampilkan Data pada Detail Poli

Buka kembali file **poli_form.dart** import halaman **poli_detail.dart** kemudian tambahkan attribute

```
final _namaPoliCtrl = TextEditingController();
```

Pada **poli_form.dart** sehingga menjadi

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import 'poli_detail.dart';
4.
5.  class PoliForm extends StatefulWidget {
6.    const PoliForm({Key? key}) : super(key: key);
7.    _PoliFormState createState() => _PoliFormState();
8.  }
9.
10. class _PoliFormState extends State<PoliForm> {
11.   final _formKey = GlobalKey<FormState>();
12.   final _namaPoliCtrl = TextEditingController();
13.
14.   @override
15.   Widget build(BuildContext context) {
16.     return Scaffold(
17.       appBar: AppBar(title: const Text("Tambah Poli")),
18.       body: SingleChildScrollView(
19.         child: Form(
20.           key: _formKey,
21.           child: Column(
22.             children: [_fieldNamaPoli(), SizedBox(height: 20), _tombolSimpan()],
23.           ),
24.         ),
25.       ),
26.     );
27.   }
28.
29.   _fieldNamaPoli() {
30.     return TextField(
31.       decoration: const InputDecoration(labelText: "Nama Poli"),
32.       controller: _namaPoliCtrl,
33.     );
34.   }
35.
36.   _tombolSimpan() {
37.     return ElevatedButton(
38.         onPressed: () {
39.           Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
40.           Navigator.pushReplacement(context,
41.               MaterialPageRoute(builder: (context) => PoliDetail(poli: poli)));
42.         },
43.         child: const Text("Simpan"));
44.   }
45. }
```

Pada setiap TextField yang telah dibuat, data yang diinput dikirim ke attribute TextEditingController() yang telah kita buat sebelumnya

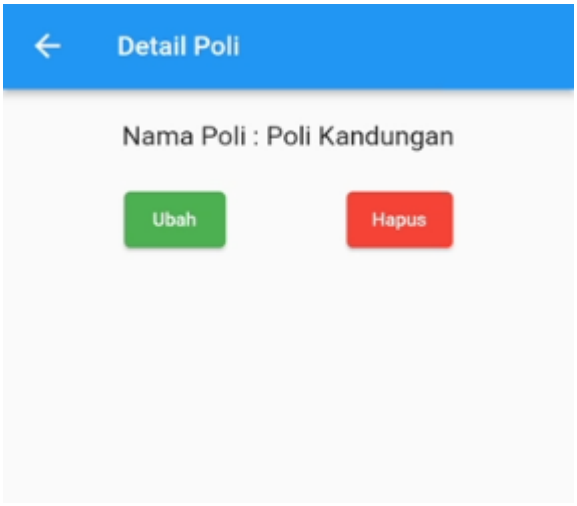
Pada fungsi **_fieldNamaPoli()** menjadi

```
_fieldNamaPoli() {
    return TextField(
      decoration: const InputDecoration(labelText: "Nama Poli"),
      controller: _namaPoliCtrl,
    );
  }
```

Kemudian pada fungsi **_tombolSimpan()** pada saat diklik akan mengirim data inputan dan menampilkan data tersebut pada **PoliDetail** yang telah kita buat sebelumnya

```
_tombolSimpan() {
    return ElevatedButton(
        onPressed: () {
          Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
          Navigator.pushReplacement(context
              MaterialPageRoute(builder: (context) => PoliDetail(poli: poli)));
        },
        child: const Text("Simpan"));
  }
```





PERTEMUAN 5

Membuat Form Ubah Poli

Pada bagian ini kita akan membuat Form untuk mengubah data poli, dimana pada halaman Detail Poli terdapat tombol ubah dan pada saat tombol ubah tersebut diklik, maka halaman akan beralih ke Form Ubah Poli dengan menampilkan data poli yang akan diubah.

Buat sebuah file dengan nama **poli_update_form.dart** di dalam folder **ui**, kemudian ketikkan kode berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import 'poli_detail.dart';
4.
5.  class PoliUpdateForm extends StatefulWidget {
6.    final Poli poli;
7.
8.    const PoliUpdateForm({Key? key, required this.poli}) : super(key: key);
9.    _PoliUpdateFormState createState() => _PoliUpdateFormState();
10. }
11.
12. class _PoliUpdateFormState extends State<PoliUpdateForm> {
13.   final _formKey = GlobalKey<FormState>();
14.   final _namaPoliCtrl = TextEditingController();
15.
16.   @override
17.   void initState() {
18.     super.initState();
19.     setState(() {
20.       _namaPoliCtrl.text = widget.poli.namaPoli;
21.     });
22.   }
23.
24.   @override
25.   Widget build(BuildContext context) {
26.     return Scaffold(
27.       appBar: AppBar(title: const Text("Ubah Poli")),
28.       body: SingleChildScrollView(
29.         child: Form(
30.           key: _formKey,
31.           child: Column(
32.             children: [_fieldNamaPoli(), SizedBox(height: 20), _tombolSimpan()],
33.           ),
34.         ),
35.       ),
36.     );
37.   }
38.
39.   _fieldNamaPoli() {
40.     return TextField(
41.       decoration: const InputDecoration(labelText: "Nama Poli"),
42.       controller: _namaPoliCtrl,
43.     );
44.   }
45.
46.   _tombolSimpan() {
47.     return ElevatedButton(
48.         onPressed: () {
49.           Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
50.           Navigator.pop(context);
51.           Navigator.pushReplacement(context,
52.               MaterialPageRoute(builder: (context) => PoliDetail(poli: poli)));
```

```
        },
        child: const Text("Simpan Perubahan"));
  }
}
```

Kemudian buka file **poli_detail.dart** , kita import **poli_update_form.dart** kemudian untuk memudahkan dalam pengkodean, lakukan pemisahan ke dalam fungsi untuk tombol ubah dan hapus. Pada bagian tombol ubah kita lakukan penambahan kode pada bagian **onPressed** seperti menjadi berikut

```
import 'package:flutter/material.dart';
import 'poli_update_form.dart';
import '../model/poli.dart';

class PoliDetail extends StatefulWidget {
  final Poli poli;

  const PoliDetail({super.key, required this.poli});

  @override
  State<PoliDetail> createState() => _PoliDetailState();
}

class _PoliDetailState extends State<PoliDetail> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Detail Poli")),
      body: Column(
        children: [
          SizedBox(height: 20),
          Text(
            "Nama Poli : ${widget.poli.namaPoli}",
            style: TextStyle(fontSize: 20),
          ),
          SizedBox(height: 20),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              _tombolUbah(),
              _tombolHapus(),
            ],
          )
        ],
      ),
    );
  }

  _tombolUbah() {
    return ElevatedButton(
        onPressed: () {
          Navigator.push(
              context,
              MaterialPageRoute(
                  builder: (context) => PoliUpdateForm(poli: widget.poli)));
        },
        style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
        child: const Text("Ubah"));
  }

  _tombolHapus() {
    return ElevatedButton(
        onPressed: () {},
        style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
```

```
        child: const Text("Hapus"));
  }
}
```

Menampilkan Pesan Konfirmasi Hapus Data

Masih pada file **poli_detail.dart** kita akan mengimport **poli_page.dart** dan juga memodifikasi fungsi **_tombolHapus** dimana saat tombol hapus diklik, akan memunculkan pesan konfirmasi penghapusan data, jika yakin ingin menghapus, halaman akan beralih ke halaman data mahasiswa. Namun untuk saat ini, karena tidak menggunakan database, maka data tidak akan terhapus.

```
_tombolHapus() {
  return ElevatedButton(
      onPressed: () {
        AlertDialog alertDialog = AlertDialog(
          content: const Text("Yakin ingin menghapus data ini?"),
          actions: [
            // tombol ya
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context);
                Navigator.pushReplacement(context,
                    MaterialPageRoute(builder: (context) => PoliPage()));
              },
              child: const Text("YA"),
              style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
            ),
            // tombol batal
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context);
              },
              child: Text("Tidak"),
              style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
            )
          ],
        );
        showDialog(context: context, builder: (context) => alertDialog);
      },
      style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
      child: const Text("Hapus"));
}
```

Adapun keseluruhan kode **poli_detail.dart** adalah

```
import 'package:flutter/material.dart';
import 'poli_page.dart';
import 'poli_update_form.dart';
import '../model/poli.dart';

class PoliDetail extends StatefulWidget {
  final Poli poli;

  const PoliDetail({super.key, required this.poli});

  @override
  State<PoliDetail> createState() => _PoliDetailState();
}

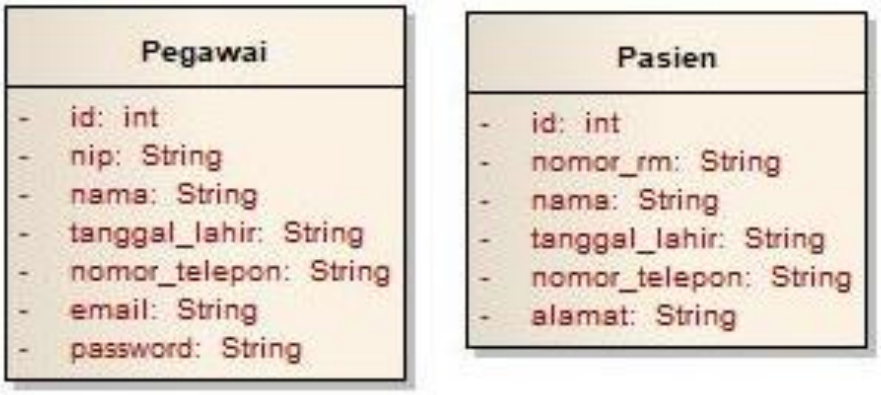
class _PoliDetailState extends State<PoliDetail> {
  @override
```

```
17.   Widget build(BuildContext context) {
18.     return Scaffold(
19.       appBar: AppBar(title: Text("Detail Poli")),
20.       body: Column(
21.         children: [
22.           SizedBox(height: 20),
23.           Text(
24.             "Nama Poli : ${widget.poli.namaPoli}",
25.             style: TextStyle(fontSize: 20),
26.           ),
27.           SizedBox(height: 20),
28.           Row(
29.             mainAxisAlignment: MainAxisAlignment.spaceEvenly,
30.             children: [
31.               _tombolUbah(),
32.               _tombolHapus(),
33.             ],
34.           )
35.         ],
36.       ),
37.     );
38.   }
39.
40.   _tombolUbah() {
41.     return ElevatedButton(
42.         onPressed: () {
43.           Navigator.push(
44.               context,
45.               MaterialPageRoute(
46.                   builder: (context) => PoliUpdateForm(poli: widget.poli)));
47.         },
48.         style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
49.         child: const Text("Ubah"));
50.   }
51.
52.   _tombolHapus() {
53.     return ElevatedButton(
54.         onPressed: () {
55.           AlertDialog alertDialog = AlertDialog(
56.             content: const Text("Yakin ingin menghapus data ini?"),
57.             actions: [
58.               // tombol ya
59.               ElevatedButton(
60.                 onPressed: () {
61.                   Navigator.pop(context);
62.                   Navigator.pushReplacement(context,
63.                       MaterialPageRoute(builder: (context) => PoliPage()));
64.                 },
65.                 child: const Text("YA"),
66.                 style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
67.               ),
68.               // tombol batal
69.               ElevatedButton(
70.                 onPressed: () {
71.                   Navigator.pop(context);
72.                 },
73.                 child: Text("Tidak"),
74.                 style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
75.               )
76.             ],
77.           );
78.           showDialog(context: context, builder: (context) => alertDialog);
79.         },
80.         style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
81.         child: const Text("Hapus"));
82.   }
```

```
83. }
```

TUGAS

Buat sebagaimana halnya pertemuan 7 (Membuat fungsi tombol simpan, menampilkan data pada halaman detail, membuat form ubah dan menampilkan pesan konfirmasi hapus) dengan skema seperti gambar dibawah



PERTEMUAN 6

Membuat Halaman Beranda

Buat sebuah file dengan nama **beranda.dart** pada folder **ui** dan ketikkan kode berikut

```
1.  import 'package:flutter/material.dart';
2.
3.  class Beranda extends StatelessWidget {
4.    const Beranda({super.key});
5.
6.    @override
7.    Widget build(BuildContext context) {
8.      return Scaffold(
9.        appBar: AppBar(title: Text("Beranda")),
10.       body: Center(
11.         child: Text("Selamat Datang"),
12.       ),
13.     );
14.   }
15. }
```

Membuat Halaman Login

Buat sebuah file dengan nama **login.dart** pada folder **ui** dan ketikkan kode berikut

```
1.  import 'package:flutter/material.dart';
2.
3.  class Login extends StatefulWidget {
4.    const Login({Key? key}) : super(key: key);
5.    _LoginState createState() => _LoginState();
6.  }
7.
8.  class _LoginState extends State<Login> {
9.    final _formKey = GlobalKey<FormState>();
10.   final _usernameCtrl = TextEditingController();
11.   final _passwordCtrl = TextEditingController();
12.
13.   @override
14.   Widget build(BuildContext context) {
15.     return Scaffold(
16.       body: SingleChildScrollView(
17.         child: SafeArea(
18.             child: Container(
19.           height: MediaQuery.of(context).size.height,
20.           child: Column(
21.             mainAxisAlignment: MainAxisAlignment.center,
22.             children: [
23.               Text("Login Admin",
24.                   style: TextStyle(fontSize: 22, fontWeight: FontWeight.w500)),
25.               SizedBox(height: 50),
26.               Center(
27.                 child: Form(
28.                     key: _formKey,
29.                     child: Container(
30.                       width: MediaQuery.of(context).size.width / 1.3,
31.                       child: Column(
32.                         children: [
33.                           _usernameTextField(),
34.                           SizedBox(height: 20),
35.                           _passwordTextField(),
36.                           SizedBox(height: 40),
```

```
                          _tombolLogin(),
                        ],
                      ),
                    )),
              )
            ],
          ),
        )),
      ),
    );
  }

  Widget _usernameTextField() {
    return TextFormField(
      decoration: InputDecoration(labelText: "Username"),
      controller: _usernameCtrl,
    );
  }

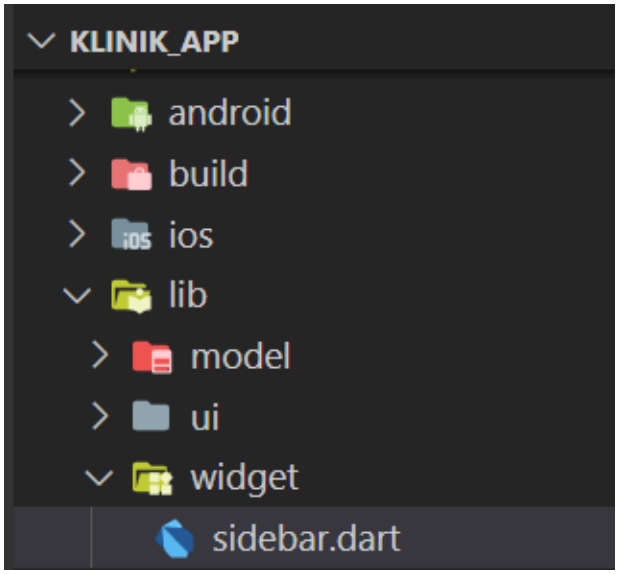
  Widget _passwordTextField() {
    return TextFormField(
      decoration: InputDecoration(labelText: "Password"),
      obscureText: true,
      controller: _passwordCtrl,
    );
  }

  Widget _tombolLogin() {
    return Container(
        width: MediaQuery.of(context).size.width,
        child: ElevatedButton(child: Text("Login"), onPressed: () {}));
  }
}
```

Membuat Sidebar Menu (Drawer)

Drawer adalah menu yang berada di sebelah kiri layar android. Biasanya drawer akan muncul saat kita menekan icon menu yang berada di kiri atas appbar. Sedangkan di website, drawer bisa kita sebut juga dengan sidebar menu.

Buat sebuah folder di dalam **lib** dengan nama **widget** kemudian di dalam folder **widget** tersebut buat sebuah file dengan nama **sidebar.dart**



Buka file **sidebar.dart** dan ketikkan kode berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../ui/beranda.dart';
3.  import '../ui/login.dart';
4.  import '../ui/poli_page.dart';
5.
6.  class Sidebar extends StatelessWidget {
7.    const Sidebar({Key? key}) : super(key: key);
8.    @override
9.    Widget build(BuildContext context) {
10.     return Drawer(
11.       child: ListView(
12.         padding: EdgeInsets.zero,
13.         children: [
14.           UserAccountsDrawerHeader(
15.               accountName: Text("Admin"),
16.               accountEmail: Text("admin@admin.com")),
17.           ListTile(
18.             leading: Icon(Icons.home),
19.             title: Text("Beranda"),
20.             onTap: () {
21.               Navigator.push(
22.                   context, MaterialPageRoute(builder: (context) => Beranda()));
23.             },
24.           ),
25.           ListTile(
26.             leading: Icon(Icons.accessible),
27.             title: Text("Poli"),
28.             onTap: () {
29.               Navigator.push(
30.                   context, MaterialPageRoute(builder: (context) => PoliPage()));
31.             },
32.           ),
33.           ListTile(
34.             leading: Icon(Icons.people),
35.             title: Text("Pegawai"),
36.             onTap: () {},
37.           ),
38.           ListTile(
39.             leading: Icon(Icons.account_box_sharp),
40.             title: Text("Pasien"),
41.             onTap: () {},
42.           ),
```

```
43.           ListTile(
44.             leading: Icon(Icons.logout_rounded),
45.             title: Text("Keluar"),
46.             onTap: () {
47.               Navigator.pushAndRemoveUntil(
48.                   context,
49.                   MaterialPageRoute(builder: (context) => Login()),
50.                   (Route<dynamic> route) => false);
51.             },
52.           )
53.         ],
54.       ),
55.     );
56.   }
57. }
```

Kemudian buka file **beranda.dart** pada bagian Scaffold tambahkan parameter **drawer** dan masukkan class **Sidebar** pada parameter drawer tersebut (baris 10).

```
1.  import 'package:flutter/material.dart';
2.  import '../widget/sidebar.dart';
3.
4.  class Beranda extends StatelessWidget {
5.    const Beranda({super.key});
6.
7.    @override
8.    Widget build(BuildContext context) {
9.      return Scaffold(
10.       drawer: Sidebar(),
11.       appBar: AppBar(title: Text("beranda")),
12.       body: Center(
13.         child: Text("Selamat Datang"),
14.       ),
15.     );
16.   }
17. }
```

Kemudian buka file **poli_page.dart** pada bagian Scaffold lakukan hal yang sama seperti pada halaman beranda yaitu tambahkan parameter **drawer** dan masukkan class **Sidebar** pada parameter drawer tersebut (baris 19).

```
1.  import 'package:flutter/material.dart';
2.  import '../widget/sidebar.dart';
3.  import '../model/poli.dart';
4.  import 'poli_detail.dart';
5.  import 'poli_item.dart';
6.  import 'poli_form.dart';
7.
8.  class PoliPage extends StatefulWidget {
9.    const PoliPage({super.key});
10.
11.   @override
12.   State<PoliPage> createState() => _PoliPageState();
13. }
14.
15. class _PoliPageState extends State<PoliPage> {
16.   @override
17.   Widget build(BuildContext context) {
```

```
    return Scaffold(
      drawer: Sidebar(),
      appBar: AppBar(
        title: const Text("Data Poli"),
        actions: [
          GestureDetector(
            child: const Icon(Icons.add),
            onTap: () {
              Navigator.push(
                  context, MaterialPageRoute(builder: (context) => PoliForm()));
            },
          )
        ],
      ),
      body: ListView(
        children: [
          PoliItem(poli: Poli(namaPoli: "Poli Anak")),
          PoliItem(poli: Poli(namaPoli: "Poli Kandungan")),
          PoliItem(poli: Poli(namaPoli: "Poli Gigi")),
          PoliItem(poli: Poli(namaPoli: "Poli THT")),
        ],
      ),
    );
  }
}
```

Buka file **main.dart** arahkan halaman awal pada beranda

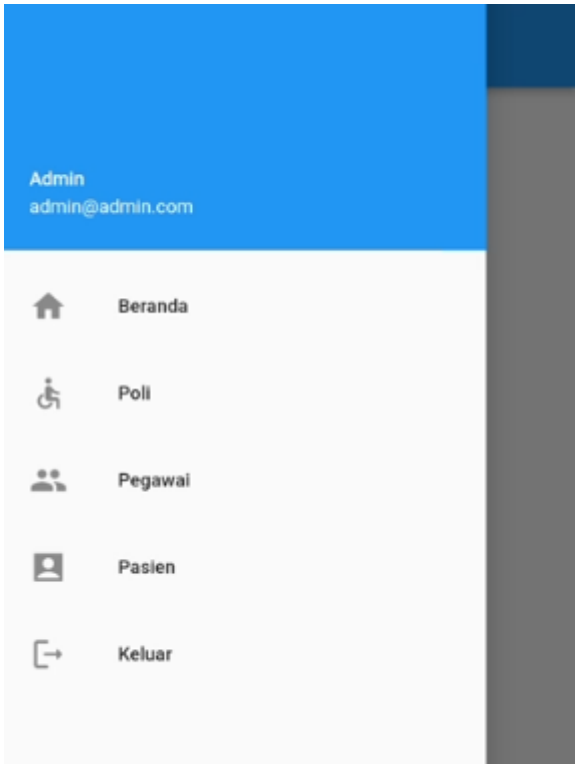
```
import 'package:flutter/material.dart';
import 'ui/beranda.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Klinik',
      home: Beranda(),
    );
  }
}
```

Sehingga hasilnya akan seperti berikut



TUGAS

- Buat Pesan Konfirmasi hapus data untuk bagian Pegawai dan Pasien
- Jika kita lihat pada drawer, terdapat menu **Beranda, Poli, Pegawai, Pasien dan** Keluar. Tugas Anda menyatukan halaman Manajemen (CRUD) Pegawai dan Pasien yang telah anda buat sebelumnya untuk tampil saat menu pegawai atau pasien dipilih

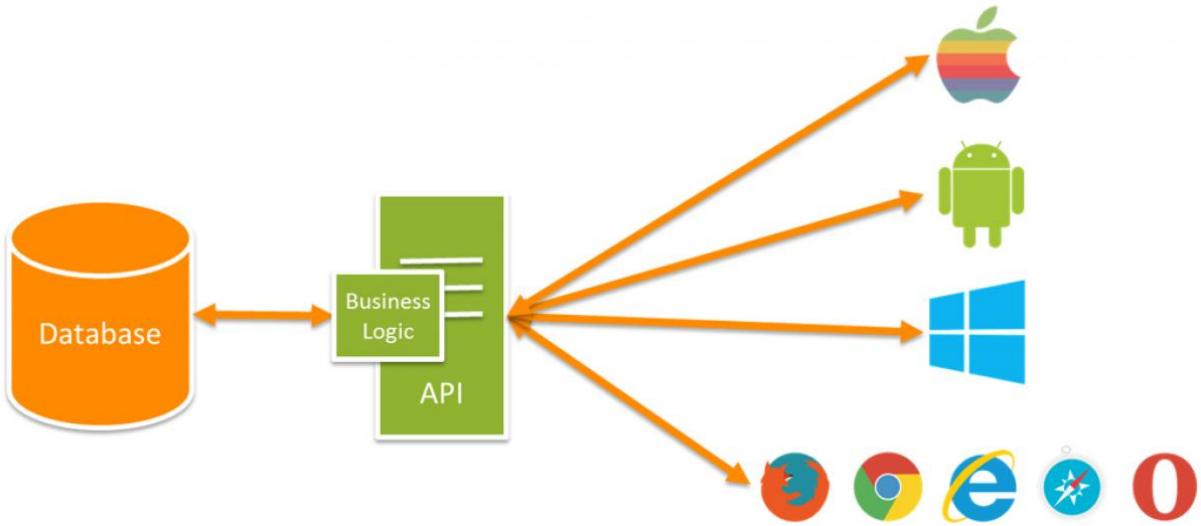
PERTEMUAN 7

Apa itu API

API atau Application Programming Interface adalah sebuah interface yang dapat menghubungkan aplikasi satu dengan aplikasi lainnya. Jadi, API berperan sebagai perantara antar berbagai aplikasi berbeda, baik dalam satu platform yang sama atau lintas platform.

Perumpamaan yang bisa digunakan untuk menjelaskan API adalah seorang pelayan di restoran. Tugas pelayan tersebut adalah menghubungkan tamu restoran dengan juru masak. Tamu cukup memesan makanan sesuai daftar menu yang ada dan pelayan memberitahukannya ke juru masak. Nantinya, pelayan akan kembali ke tamu tadi dengan masakan yang sudah siap sesuai pesanan.

Itulah gambaran tugas dari API dalam pengembangan aplikasi.



Sumber : codepolitan.com

Arsitektur API

Ada tiga arsitektur API yang sering digunakan oleh developer dalam pembangunan aplikasi. Arsitektur ini berkaitan pada bentuk data yang dikirim. Adapun Arsitektur API yang sering digunakan adalah

1. RPC

RPC merupakan teknologi untuk membuat komunikasi antara client side dan server side bisa dilakukan dengan konsep sederhana.

RPC memiliki dua jenis, yaitu XML-RPC dan JSON-RPC. Sesuai namanya, XML-RPC menggunakan format XML sebagai media perpindahan data, sedangkan JSON-RPC menggunakan JSON untuk perpindahan data.

2. SOAP

Arsitektur API lainnya adalah SOAP (Simple Object Access Protocol). Arsitektur ini menggunakan XML (Extensible Markup Language) yang memungkinkan semua data disimpan dalam dokumen.

3. REST

REST atau Representational State Transfer adalah arsitektur API yang cukup populer karena kemudahan penggunaannya. Tak perlu coding yang panjang untuk menggunakannya.

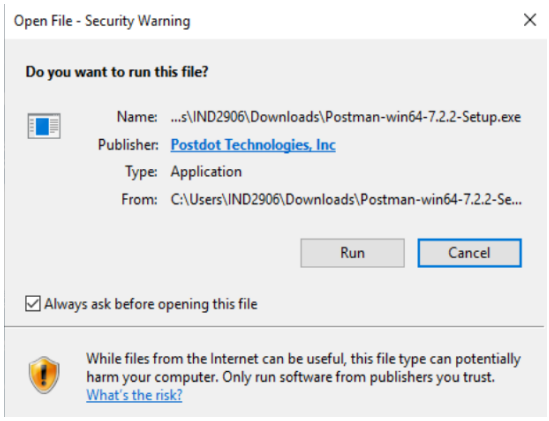
REST menggunakan JSON sebagai bentuk datanya sehingga lebih ringan. Performa aplikasi pun menjadi lebih baik.

Install Postman

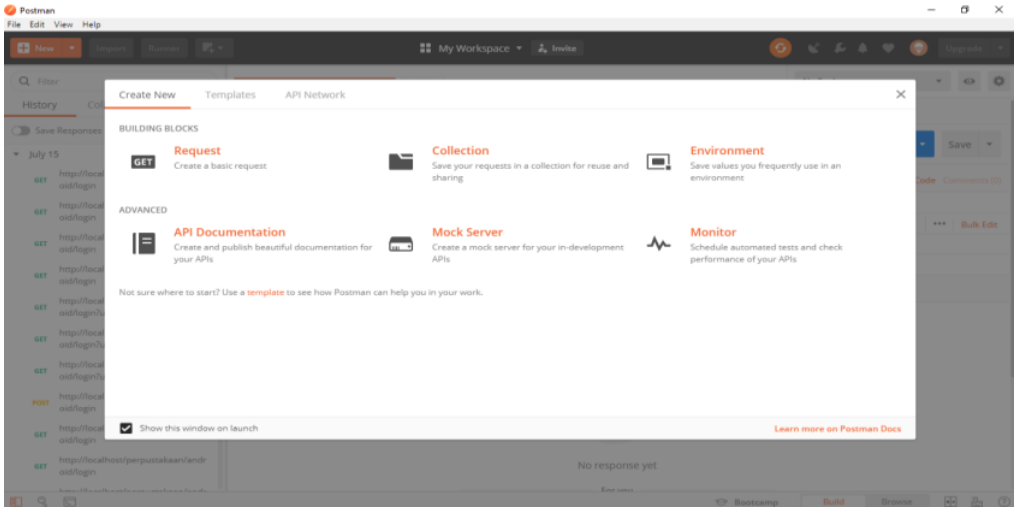
Postman adalah sebuah aplikasi fungsinya adalah sebagai REST Client atau istilahnya adalah aplikasi yang digunakan untuk melakukan uji coba REST API yang telah kita buat. Postman ini merupakan tools wajib bagi para developer yang bergerak pada pembuatan API, fungsi utama postman ini adalah sebagai GUI API Caller Pemanggil. namun sekarang postman juga menyediakan fitur lain yaitu Sharing Collection API for Documentation (free), Testing API (free), Realtime Collaboration Team (paid), Monitoring API (paid), Integration (paid).

Postman tersedia sebagai aplikasi asli untuk sistem operasi macOS, Windows (32-bit dan 64-bit), dan Linux (32-bit dan 64-bit). Untuk mendapatkan aplikasi Postman, dapat diunduh pada website resminya yanitu getpostman.com atau dapat diunduk pada halaman https://www.postman.com/downloads/

Setelah berhasil mengunduh paket instalasi postman, kemudian jalankan dengan cara klik dua kali. Pilih run jika muncul pop up seperti berikut :



Kemudian tunggu hingga proses instalasi selesai dan muncul seperti gambar berikut


Postman
File Edit View Help
New
Import
Runner
My Workspace
Invite
Upgrade
History
Save Responses
July 15
Save
Create New
Templates
API Network
BUILDING BLOCKS
Request
Create a basic request
Collection
Save your requests in a collection for reuse and sharing
Environment
Save values you frequently use in an environment
ADVANCED
API Documentation
Create and publish beautiful documentation for your APIs
Mock Server
Create a mock server for your in-development APIs
Monitor
Schedule automated tests and check performance of your APIs
Not sure where to start? Use a template to see how Postman can help you in your work.
Show this window on launch
Learn more on Postman Docs
No response yet
Bootcamp
Build
Browse

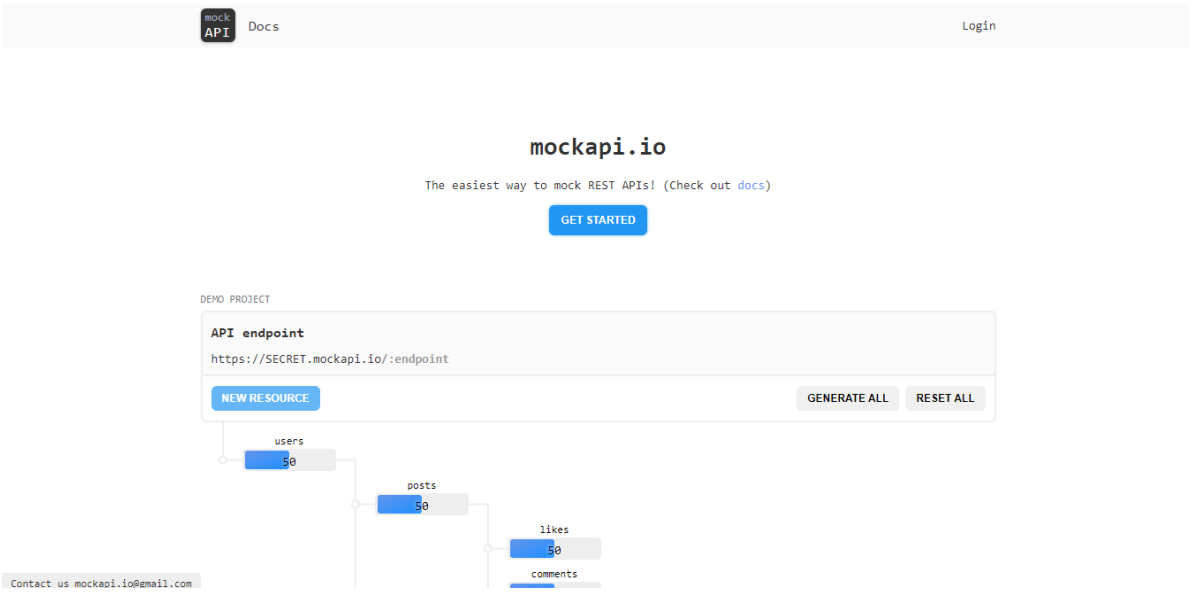
PERTEMUAN 8

UJIAN TENGAH SEMESTER

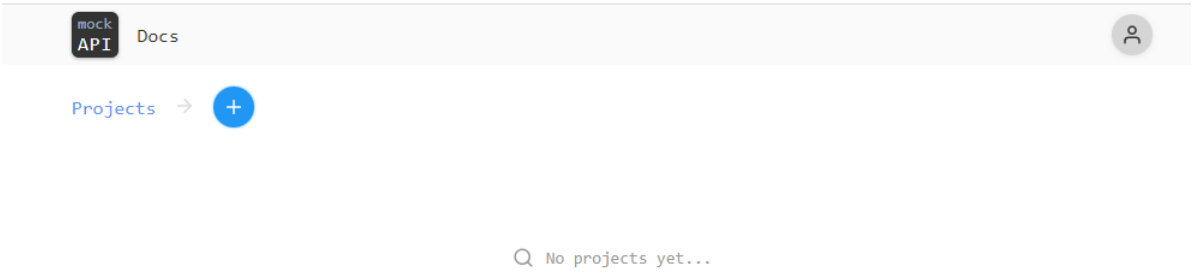
PERTEMUAN 9

Membuat API Klinik dummy dengan mockapi.io

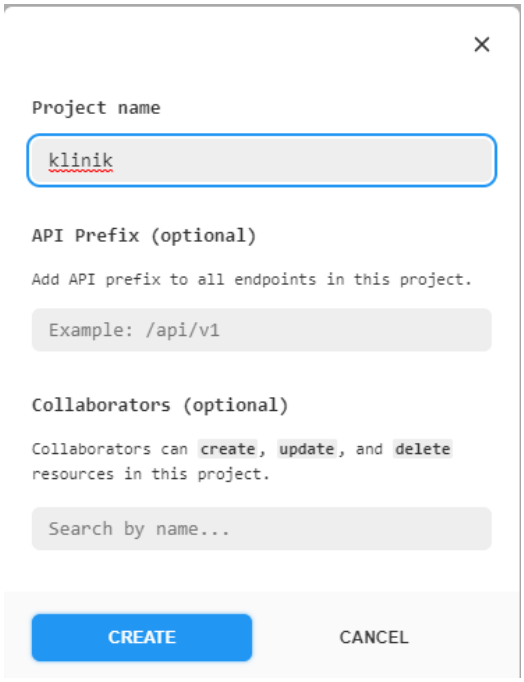
Silahkan kunjungi laman https://mockapi.io/ kemudian daftar/buat akun mockapi.io

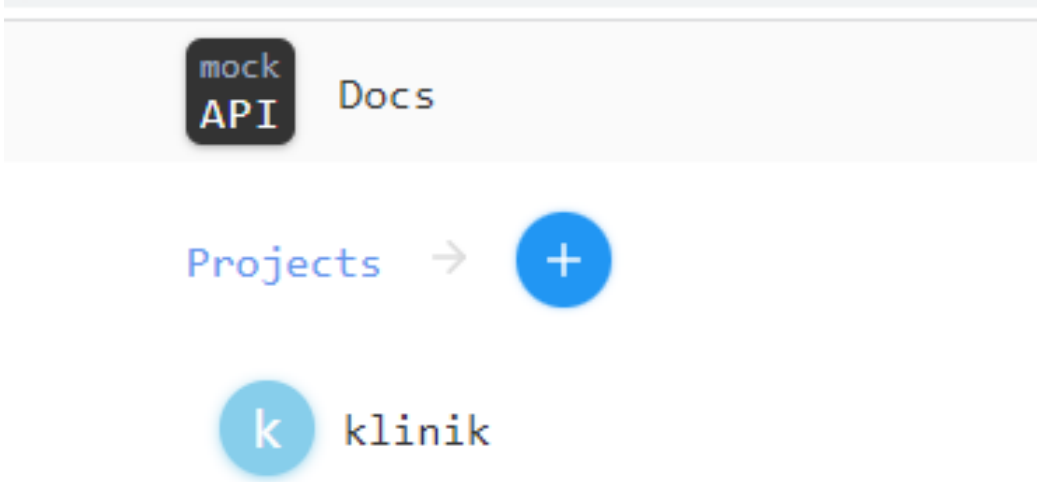


Jika sudah login ke dalam mockapi.io kemudian buat projek dengan mengklik tombol tambah

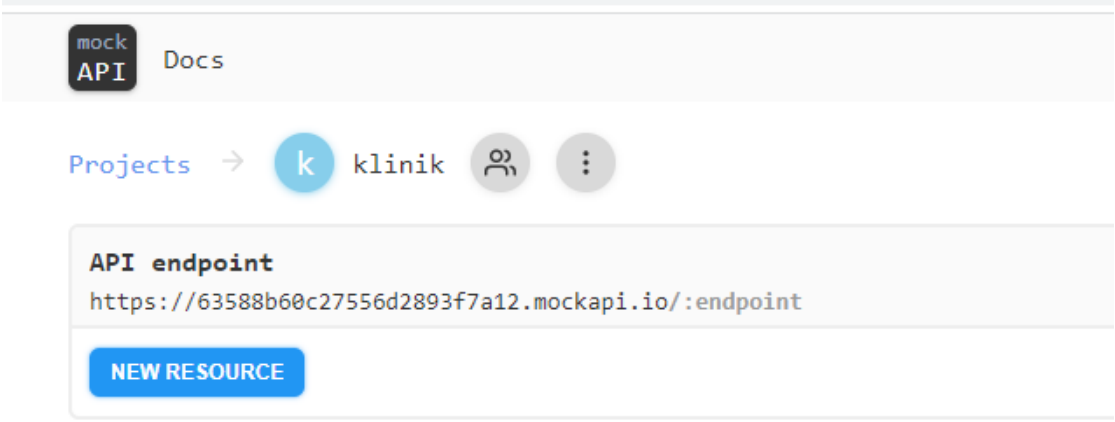


Buat projek dengan nama **klinik** kemudian klik tombol **create**

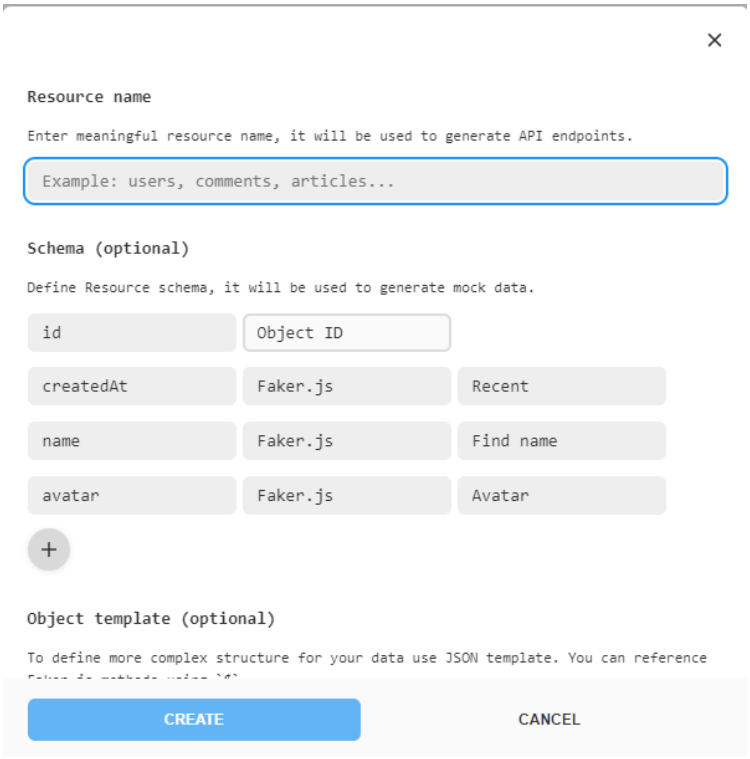

Project name
klinik
API Prefix (optional)
Add API prefix to all endpoints in this project.
Example: /api/v1
Collaborators (optional)
Collaborators can create, update, and delete resources in this project.
Search by name...
CREATE
CANCEL


mock API
Docs
Projects
k
klinik

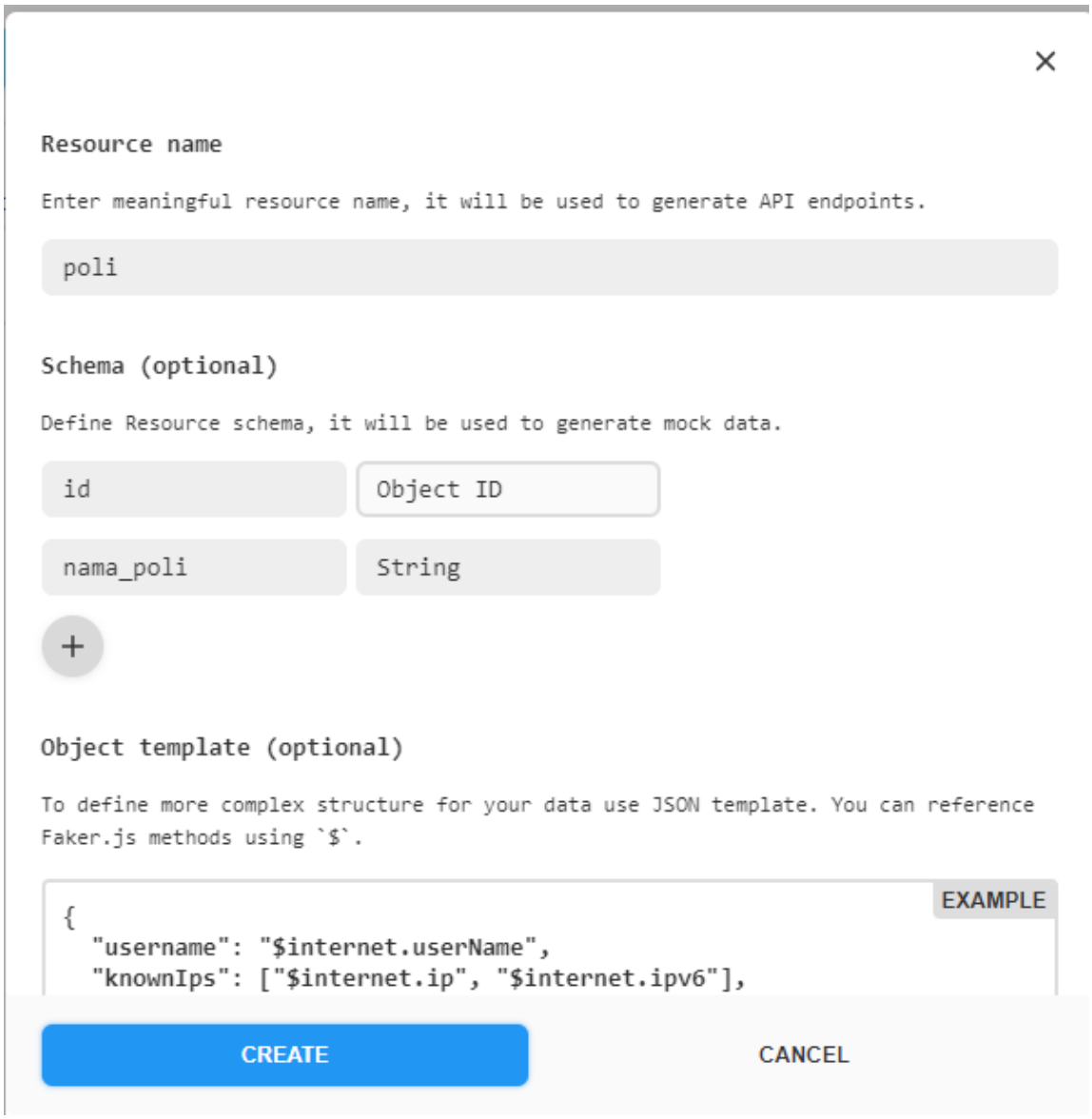
Kemudian klik projek klinik yang telah dibuat tadi sehingga muncul seperti tampilan berikut



Klik tombol **New Resource** sehingga muncul tampilan berikut



Isi **Resourse Name** dengan **poli** kemudian pada bagian **Schema** ubah sehingga menjadi seperti gambar berikut



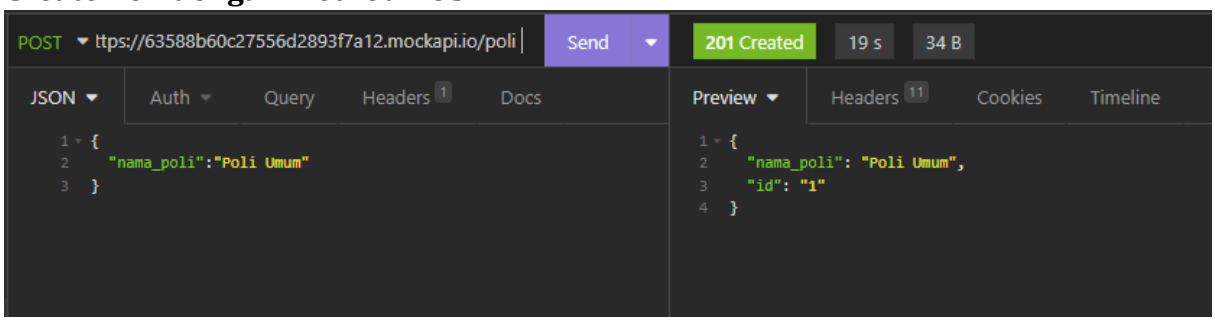
Kemudian klik tombol **create** dan hasilnya akan menjadi seperti berikut



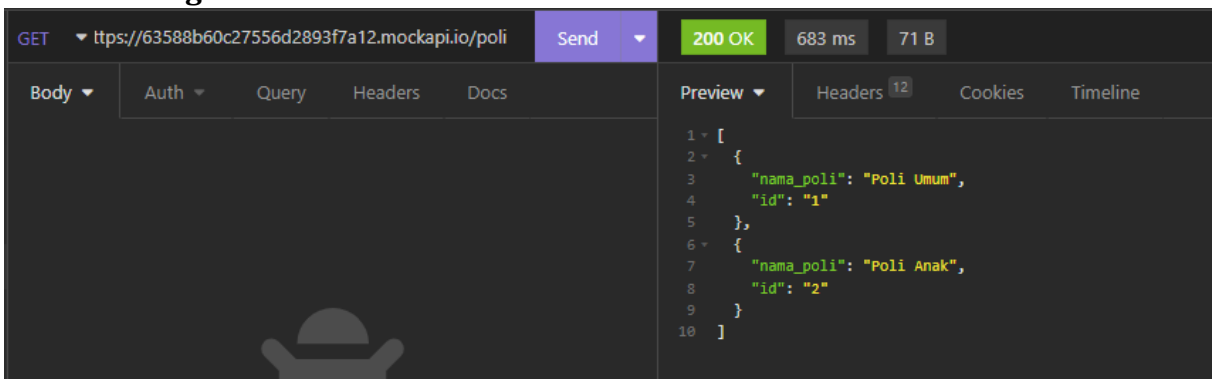
Pada bagian bawah **API endpoint** (yang digaris bawah merah) merupakan alamat utama url yang dapat kita kunjungi untuk mengakses API.

Untuk mengakses data mahasiswa yang telah kita buat sebelumnya, kita dapat mengakses dengan alamat `https://63588b60c27556d2893f7a12.mockapi.io/poli` (sesuaikan dengan url yang ada pada mockapi anda)

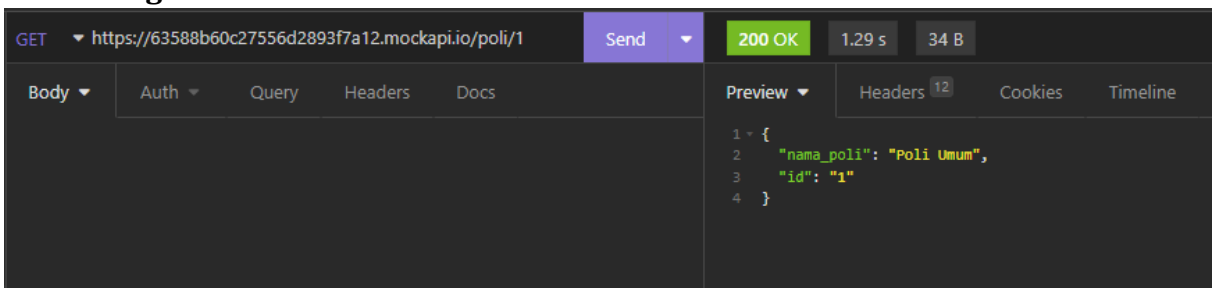
Create Poli dengan method POST



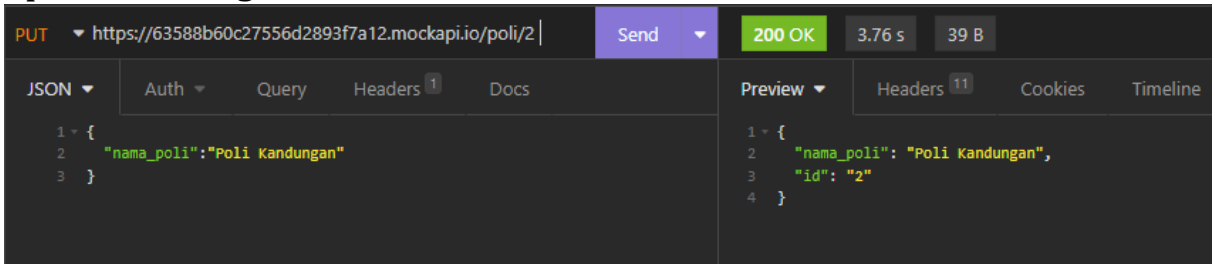
List Poli dengan method GET



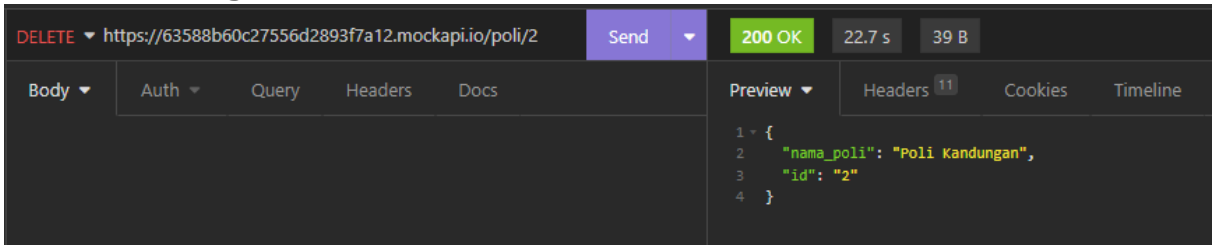
Show dengan method GET

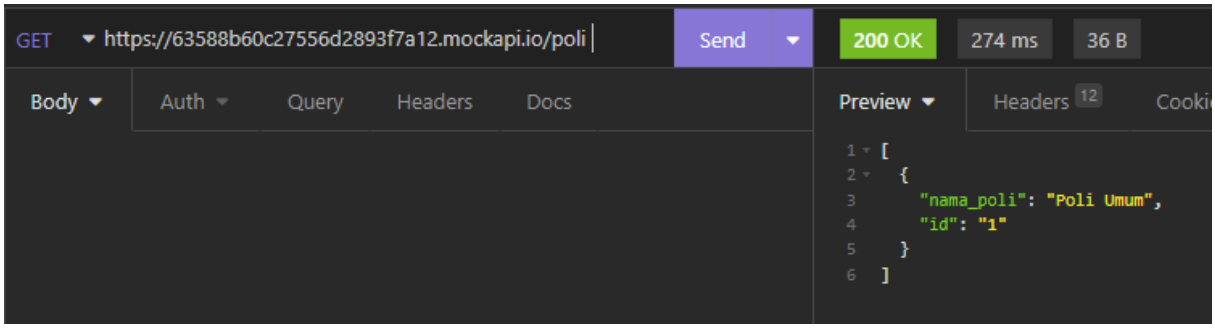


Update Poli dengan method PUT



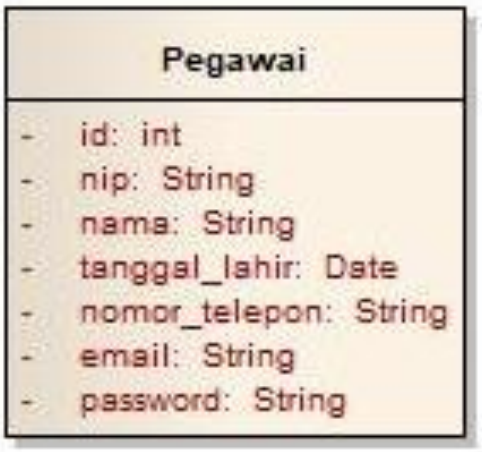
Delete Poli dengan method DELETE

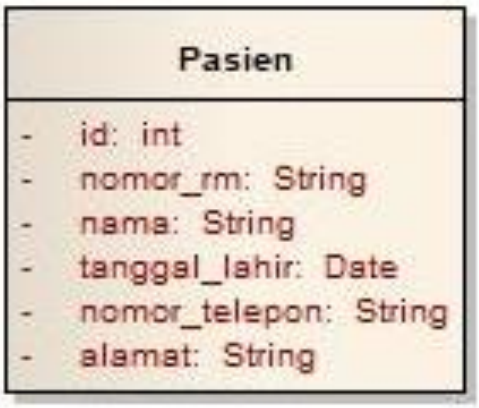




TUGAS

Buat tabel (Resource) dengan skema seperti berikut





PERTEMUAN 10

Menambahkan Depedencies

Dalam pembuatan aplikasi ini dibutuhkan depedensi untuk menerima dan mengirim request ke Rest API (**dio**). Pastikan komputer atau laptop terhubung ke internet, buka file **pubspec.yaml** kemudian pada bagian **dependencies** tambahkan kode berikut

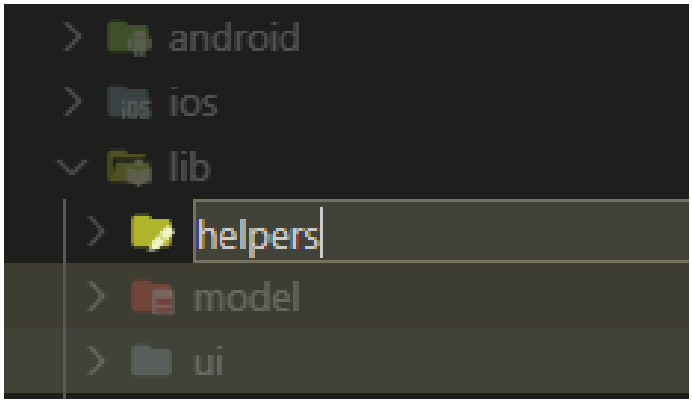
```
29. dependencies:
30.   flutter:
31.     sdk: flutter
32.
33.
34.   # The following adds the Cupertino Icons font to your application.
35.   # Use with the CupertinoIcons class for iOS style icons.
36.   cupertino_icons: ^1.0.2
37.   dio: ^5.6.0
38.   shared_preferences: ^2.3.2
39.
40. dev_dependencies:
41.   flutter_test:
42.     sdk: flutter
```

Simpan file tersebut, dan secara otomatis VS Code akan mengunduh depedencies tersebut, jika tidak berhasil, untuk mengunduh depedencies atau package yang telah ditambahkan, dapat dilakukan dengan membuka **CommandPrompt** kemudian masuk ke folder projek dan ketikkan **flutter pub get** kemudian tekan Enter

```
[perpustakaan_app] flutter pub get
Running "flutter pub get" in perpustakaan_app...
exit code 0
```

Membuat Class Untuk Menyimpan Token

Pada bagian ini kita akan membuat class yang akan menyimpan token dan data pengguna saat berhasil login. Buat sebuah folder dengan nama helpers



Kemudian pada folder helpers buat sebuah file dengan nama **user_info.dart** dan masukkan kode berikut

```
1.  import 'package:shared_preferences/shared_preferences.dart';
2.
3.  const String TOKEN = "token";
4.  const String USER_ID = "userID";
```

```
5.  const String USERNAME = "username";
6.
7.  class UserInfo {
8.    Future setToken(String value) async {
9.      final SharedPreferences pref = await SharedPreferences.getInstance();
10.     return pref.setString(TOKEN, value);
11.   }
12.
13.   Future<String?> getToken() async {
14.     final SharedPreferences pref = await SharedPreferences.getInstance();
15.     return pref.getString(TOKEN);
16.   }
17.
18.   Future setUserID(String value) async {
19.     final SharedPreferences pref = await SharedPreferences.getInstance();
20.     return pref.setString(USER_ID, value);
21.   }
22.
23.   Future<String> getUserID() async {
24.     final SharedPreferences pref = await SharedPreferences.getInstance();
25.     return pref.getString(USER_ID).toString();
26.   }
27.
28.   Future setUsername(String value) async {
29.     final SharedPreferences pref = await SharedPreferences.getInstance();
30.     return pref.setString(USERNAME, value);
31.   }
32.
33.   Future<String> getUsername() async {
34.     final SharedPreferences pref = await SharedPreferences.getInstance();
35.     return pref.getString(USERNAME).toString();
36.   }
37.
38.   Future<void> logout() async {
39.     final SharedPreferences pref = await SharedPreferences.getInstance();
40.     pref.clear();
41.   }
42. }
```

Mengarahkan Halaman Awal ke Login

Buka kembali file **main.dart** kita akan memodifikasi file tersebut dengan kondisi jika belum login maka akan membuka halaman login, namun jika sudah login maka akan membuka halaman beranda

```
1.  import 'package:flutter/material.dart';
2.  import '/helpers/user_info.dart';
3.  import '/ui/beranda.dart';
4.  import '/ui/login.dart';
5.
6.  Future<void> main() async {
7.    WidgetsFlutterBinding.ensureInitialized();
8.    var token = await UserInfo().getToken();
9.    print(token);
10.   runApp(MaterialApp(
11.     title: "Klinik APP",
12.     debugShowCheckedModeBanner: false,
13.     home: token == null ? Login() : Beranda(),
14.   ));
15. }
```

PERTEMUAN 11

Persiapan Koneksi ke API

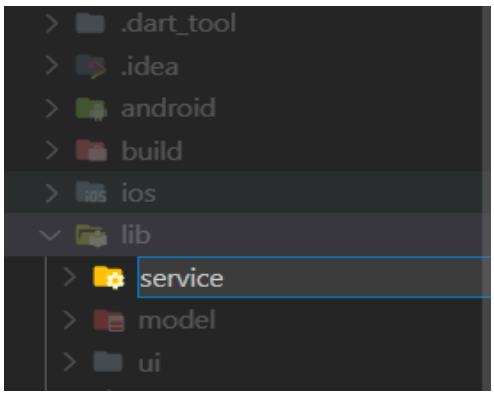
Buat sebuah fie dengan nama **api_client.dart** pada folder **helpers** kemudian ketikkan kode berikut

```
1. import 'package:dio/dio.dart';
2.
3. final Dio dio = Dio(BaseOptions(
4.     baseUrl: 'https://63588b60c27556d2893f7a12.mockapi.io/',
5.     connectTimeout: Duration(seconds: 5),
6.     receiveTimeout: Duration(seconds: 3)));
7.
8. class ApiClient {
9.   Future<Response> get(String path) async {
10.    try {
11.      final response = await dio.get(Uri.encodeFull(path));
12.      return response;
13.    } on DioError catch (e) {
14.      throw Exception(e.message);
15.    }
16.  }
17.
18.  Future<Response> post(String path, dynamic data) async {
19.    try {
20.      final response = await dio.post(Uri.encodeFull(path), data: data);
21.      return response;
22.    } on DioError catch (e) {
23.      throw Exception(e.message);
24.    }
25.  }
26.
27.  Future<Response> put(String path, dynamic data) async {
28.    try {
29.      final response = await dio.put(Uri.encodeFull(path), data: data);
30.      return response;
31.    } on DioError catch (e) {
32.      throw Exception(e.message);
33.    }
34.  }
35.
36.  Future<Response> delete(String path) async {
37.    try {
38.      final response = await dio.delete(Uri.encodeFull(path));
39.      return response;
40.    } on DioError catch (e) {
41.      throw Exception(e.message);
42.    }
43.  }
44.}
```

Pada baris 4 (`baseUrl: 'https://63588b60c27556d2893f7a12.mockapi.io/'`) Sesuaikan dengan API Endpoint pada mockapi.io yang telah dibuat sebelumnya

Membuat Service Login

Buat sebuah folder dengan nama **service** pada folder **lib**



Pada bagian ini kita akan membuat dummy login. Buat sebuah file pada folder **service** dengan nama **login_service.dart** kemudian ketikkan kode berikut

```
1.  import '../helpers/user_info.dart';
2.
3.  class LoginService {
4.    Future<bool> login(String username, String password) async {
5.      bool isLogin = false;
6.      if (username == 'admin' && password == 'admin') {
7.        await UserInfo().setToken("admin");
8.        await UserInfo().setUserID("1");
9.        await UserInfo().setUsername("admin");
10.       isLogin = true;
11.     }
12.     return isLogin;
13.   }
14. }
```

Kemudian buka file **login.dart** yang ada pada folder **ui**. Pada bagian fungsi tombol login kita ubah menjadi seperti berikut

```
Widget _tombolLogin() {
    return Container(
        width: MediaQuery.of(context).size.width,
        child: ElevatedButton(
            child: Text("Login"),
            onPressed: () async {
              String username = _usernameCtrl.text;
              String password = _passwordCtrl.text;
              await LoginService().login(username, password).then((value) {
                if (value == true) {
                  Navigator.pushReplacement(context,
                      MaterialPageRoute(builder: (context) => Beranda()));
                } else {
                  AlertDialog alertDialog = AlertDialog(
                    content: const Text("Username atau Password Tidak Valid"),
                    actions: [
                      ElevatedButton(
                        onPressed: () {
                          Navigator.pop(context);
                        },
                        child: const Text("OK"),
                        style: ElevatedButton.styleFrom(primary: Colors.green),
```

```
                    )
                  ],
                );
                showDialog(
                    context: context, builder: (context) => alertDialog);
              }
            });
          }));
  }
```

Membuat Service Poli

Buka file **poli.dart** pada folder **model**, kemudian kita akan menambahkan dua buah method yaitu

```
factory Poli.fromJson(Map<String, dynamic> json) =>
      Poli(id: json["id"], namaPoli: json["nama_poli"]);
```

Dan

```
Map<String, dynamic> toJson() => {"nama_poli": namaPoli};
```

Sehingga secara keseluruhan kode menjadi

```
1.  class Poli {
2.    String? id;
3.    String namaPoli;
4.
5.    Poli({this.id, required this.namaPoli});
6.
7.    factory Poli.fromJson(Map<String, dynamic> json) =>
8.        Poli(id: json["id"], namaPoli: json["nama_poli"]);
9.
10.   Map<String, dynamic> toJson() => {"nama_poli": namaPoli};
11. }
```

Kemudian buat sebuah file pada folder **service** dengan nama **poli_service.dart** dan ketikkan kode berikut

```
1.  import 'package:dio/dio.dart';
2.  import '../helpers/api_client.dart';
3.  import '../model/poli.dart';
4.
5.  class PoliService {
6.    Future<List<Poli>> listData() async {
7.      final Response response = await ApiClient().get('poli');
8.      final List data = response.data as List;
9.      List<Poli> result = data.map((json) => Poli.fromJson(json)).toList();
10.     return result;
11.   }
12.
13.   Future<Poli> simpan(Poli poli) async {
14.     var data = poli.toJson();
```

```
15.     final Response response = await ApiClient().post('poli', data);
16.     Poli result = Poli.fromJson(response.data);
17.     return result;
18.   }
19.
20.   Future<Poli> ubah(Poli poli, String id) async {
21.     var data = poli.toJson();
22.     final Response response = await ApiClient().put('poli/${id}', data);
23.     print(response);
24.     Poli result = Poli.fromJson(response.data);
25.     return result;
26.   }
27.
28.   Future<Poli> getById(String id) async {
29.     final Response response = await ApiClient().get('poli/${id}');
30.     Poli result = Poli.fromJson(response.data);
31.     return result;
32.   }
33.
34.   Future<Poli> hapus(Poli poli) async {
35.     final Response response = await ApiClient().delete('poli/${poli.id}');
36.     Poli result = Poli.fromJson(response.data);
37.     return result;
38.   }
39. }
```

TUGAS

Buat kode untuk Service Pegawai dan Pasien

PERTEMUAN 12

Mengambil List Poli dari API

Buka kembali file **poli_page.dart** yang ada pada folder **ui** , kita akan menambahkan fungsi

```
Stream<List<Poli>> getList() async* {
       List<Poli> data = await PoliService().listData();
       yield data;
}
```

Dan melakukan beberapa perubahan pada bagian **body** menjadi

```
body: StreamBuilder(
        stream: getList(),
        builder: (context, AsyncSnapshot snapshot) {
          if (snapshot.hasError) {
            return Text(snapshot.error.toString());
          }
          if (snapshot.connectionState != ConnectionState.done) {
            return Center(
              child: CircularProgressIndicator(),
            );
          }
          if (!snapshot.hasData &&
              snapshot.connectionState == ConnectionState.done) {
            return Text('Data Kosong');
          }

          return ListView.builder(
            itemCount: snapshot.data.length,
            itemBuilder: (context, index) {
              return PoliItem(poli: snapshot.data[index]);
            },
          );
        },
      ),
```

Adapun modifikasi keseluruhan menjadi seperti kode berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import '../service/poli_service.dart';
4.  import 'poli_detail.dart';
5.  import 'poli_form.dart';
6.  import 'poli_item.dart';
7.  import '../widget/sidebar.dart';
8.
9.  class PoliPage extends StatefulWidget {
10.   const PoliPage({Key? key}) : super(key: key);
11.   _PoliPageState createState() => _PoliPageState();
12. }
13.
14. class _PoliPageState extends State<PoliPage> {
15.   Stream<List<Poli>> getList() async* {
16.     List<Poli> data = await PoliService().listData();
17.     yield data;
18.   }
19.
20.   @override
21.   Widget build(BuildContext context) {
22.     return Scaffold(
```

```
      drawer: Sidebar(),
      appBar: AppBar(
        title: const Text("Data Poli"),
        actions: [
          GestureDetector(
            child: const Icon(Icons.add),
            onTap: () {
              Navigator.push(
                  context,     MaterialPageRoute(builder:     (context)     =>
  PoliForm()));
            },
          )
        ],
      ),
      body: StreamBuilder(
        stream: getList(),
        builder: (context, AsyncSnapshot snapshot) {
          if (snapshot.hasError) {
            return Text(snapshot.error.toString());
          }
          if (snapshot.connectionState != ConnectionState.done) {
            return Center(
              child: CircularProgressIndicator(),
            );
          }
          if (!snapshot.hasData &&
              snapshot.connectionState == ConnectionState.done) {
            return Text('Data Kosong');
          }

          return ListView.builder(
            itemCount: snapshot.data.length,
            itemBuilder: (context, index) {
              return PoliItem(poli: snapshot.data[index]);
            },
          );
        },
      ),
    );
  }
}
```

Menambah Data Poli ke API

Buka file **poli_form.dart** pada folder **ui** pada bagian fungsi _tombolSimpan() kita ubah menjadi seperti berikut

```
_tombolSimpan() {
    return ElevatedButton(
        onPressed: () async {
          Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
          await PoliService().simpan(poli).then((value) {
            Navigator.pushReplacement(
                context,
                MaterialPageRoute(
                    builder: (context) => PoliDetail(poli: value)));
          });
        },
        child: const Text("Simpan"));
  }
```

Adapun secara keseluruhan kodenya seperti berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import '../service/poli_service.dart';
4.  import 'poli_detail.dart';
5.
6.  class PoliForm extends StatefulWidget {
7.    const PoliForm({Key? key}) : super(key: key);
8.    _PoliFormState createState() => _PoliFormState();
9.  }
10.
11. class _PoliFormState extends State<PoliForm> {
12.   final _formKey = GlobalKey<FormState>();
13.   final _namaPoliCtrl = TextEditingController();
14.
15.   @override
16.   Widget build(BuildContext context) {
17.     return Scaffold(
18.       appBar: AppBar(title: const Text("Tambah Poli")),
19.       body: SingleChildScrollView(
20.         child: Form(
21.           key: _formKey,
22.           child: Column(
23.             children:      [_fieldNamaPoli(),      SizedBox(height:      20),
    _tombolSimpan()],
24.           ),
25.         ),
26.       ),
27.     );
28.   }
29.
30.   _fieldNamaPoli() {
31.     return TextField(
32.       decoration: const InputDecoration(labelText: "Nama Poli"),
33.       controller: _namaPoliCtrl,
34.     );
35.   }
```

```
36.
37.   _tombolSimpan() {
38.     return ElevatedButton(
39.         onPressed: () async {
40.           Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
41.           await PoliService().simpan(poli).then((value) {
42.             Navigator.pushReplacement(
43.                 context,
44.                 MaterialPageRoute(
45.                     builder: (context) => PoliDetail(poli: value)));
46.           });
47.         },
48.         child: const Text("Simpan"));
49.   }
50. }
```

TUGAS

Buat pula mengambil List serta menyimpan data ke API untuk tabel pegawai dan pasien

PERTEMUAN 13

Mengambil Detail Poli dari API

Buka file **poli_detail.dart** pada folder **ui**, kita akan menambahkan fungsi

```
Stream<Poli> getData() async* {
    Poli data = await PoliService().getById(widget.poli.id.toString());
    yield data;
  }
```

Dan melakukan perubahan pada bagian body menjadi

```
body: StreamBuilder(
        stream: getData(),
        builder: (context, AsyncSnapshot snapshot) {
          if (snapshot.hasError) {
            return Text(snapshot.error.toString());
          }
          if (snapshot.connectionState != ConnectionState.done) {
            return Center(
              child: CircularProgressIndicator(),
            );
          }
          if (!snapshot.hasData &&
              snapshot.connectionState == ConnectionState.done) {
            return Text('Data Tidak Ditemukan');
          }
          return Column(
            children: [
              SizedBox(height: 20),
              Text(
                "Nama Poli : ${snapshot.data.namaPoli}",
                style: TextStyle(fontSize: 20),
              ),
              SizedBox(height: 20),
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: [_tombolUbah(), _tombolHapus()],
              )
            ],
          );
        },
      ),
```

Pada fungsi _tombolUbah() menjadi

```
_tombolUbah() {
    return StreamBuilder(
        stream: getData(),
        builder: (context, AsyncSnapshot snapshot) => ElevatedButton(
            onPressed: () {
              Navigator.push(
                  context,
                  MaterialPageRoute(
                      builder: (context) =>
                          PoliUpdateForm(poli: snapshot.data)));
            },
            style: ElevatedButton.styleFrom(primary: Colors.green),
            child: const Text("Ubah")));
  }
```

Sehingga secara keseluruhan kodenya menjadi seperti berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../service/poli_service.dart';
3.  import 'poli_page.dart';
4.  import 'poli_update_form.dart';
5.  import '../model/poli.dart';
6.
7.  class PoliDetail extends StatefulWidget {
8.    final Poli poli;
9.
10.   const PoliDetail({Key? key, required this.poli}) : super(key: key);
11.   _PoliDetailState createState() => _PoliDetailState();
12. }
13.
14. class _PoliDetailState extends State<PoliDetail> {
15.   Stream<Poli> getData() async* {
16.     Poli data = await PoliService().getById(widget.poli.id.toString());
17.     yield data;
18.   }
19.
20.   @override
21.   Widget build(BuildContext context) {
22.     return Scaffold(
23.       appBar: AppBar(title: Text("Detail Poli")),
24.       body: StreamBuilder(
25.         stream: getData(),
26.         builder: (context, AsyncSnapshot snapshot) {
27.           if (snapshot.hasError) {
28.             return Text(snapshot.error.toString());
29.           }
30.           if (snapshot.connectionState != ConnectionState.done) {
31.             return Center(
32.               child: CircularProgressIndicator(),
33.             );
34.           }
35.           if (!snapshot.hasData &&
36.               snapshot.connectionState == ConnectionState.done) {
37.             return Text('Data Tidak Ditemukan');
38.           }
39.           return Column(
40.             children: [
41.               SizedBox(height: 20),
42.               Text(
43.                 "Nama Poli : ${snapshot.data.namaPoli}",
44.                 style: TextStyle(fontSize: 20),
45.               ),
46.               SizedBox(height: 20),
47.               Row(
48.                 mainAxisAlignment: MainAxisAlignment.spaceEvenly,
49.                 children: [_tombolUbah(), _tombolHapus()],
50.               )
51.             ],
52.           );
53.         },
54.       ),
55.     );
56.   }
57.
58.   _tombolUbah() {
59.     return StreamBuilder(
60.         stream: getData(),
61.         builder: (context, AsyncSnapshot snapshot) => ElevatedButton(
62.             onPressed: () {
63.               Navigator.push(
```

```
                    context,
                    MaterialPageRoute(
                        builder: (context) =>
                            PoliUpdateForm(poli: snapshot.data)));
              },
              style: ElevatedButton.styleFrom(primary: Colors.green),
              child: const Text("Ubah")));
    }

    _tombolHapus() {
      return ElevatedButton(
          onPressed: () {
            AlertDialog alertDialog = AlertDialog(
              content: const Text("Yakin ingin menghapus data ini?"),
              actions: [
                ElevatedButton(
                  onPressed: (){
                    Navigator.pop(context);
                    Navigator.pushReplacement(context, MaterialPageRoute(builder:
    (context) => PoliPage()));
                  },
                  child: const Text("YA"),
                  style: ElevatedButton.styleFrom(primary: Colors.red),
                  ),
                ElevatedButton(
                  onPressed: () {
                    Navigator.pop(context);
                  },
                  child: const Text("Tidak"),
                  style: ElevatedButton.styleFrom(primary: Colors.green),
                )
              ],
            );
            showDialog(context: context, builder: (context) => alertDialog);
          },
          style: ElevatedButton.styleFrom(primary: Colors.red),
          child: const Text("Hapus"));
          }
        }
```

Menghapus Data Poli dari API

Masih pada file **poli_detail.dart** pada folder **ui**, kita akan mengubah fungsi pada _tombolHapus menjadi

```
_tombolHapus() {
    return ElevatedButton(
        onPressed: () {
          AlertDialog alertDialog = AlertDialog(
            content: const Text("Yakin ingin menghapus data ini?"),
            actions: [
              StreamBuilder(
                  stream: getData(),
                  builder: (context, AsyncSnapshot snapshot) => ElevatedButton(
                        onPressed: () async {
                          await PoliService()
                              .hapus(snapshot.data)
                              .then((value) {
                            Navigator.pop(context);
                            Navigator.pushReplacement(
                                context,
                                MaterialPageRoute(
                                    builder: (context) => PoliPage()));
                          });
```

```
                    },
                    child: const Text("YA"),
                    style: ElevatedButton.styleFrom(primary: Colors.red),
                  )),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context);
              },
              child: const Text("Tidak"),
              style: ElevatedButton.styleFrom(primary: Colors.green),
            )
          ],
        );
        showDialog(context: context, builder: (context) => alertDialog);
      },
      style: ElevatedButton.styleFrom(primary: Colors.red),
      child: const Text("Hapus"));
}
```

Sehingga secara keseluruhan kodenya menjadi seperti berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../service/poli_service.dart';
3.  import 'poli_page.dart';
4.  import 'poli_update_form.dart';
5.  import '../model/poli.dart';
6.
7.  class PoliDetail extends StatefulWidget {
8.    final Poli poli;
9.
10.   const PoliDetail({Key? key, required this.poli}) : super(key: key);
11.   _PoliDetailState createState() => _PoliDetailState();
12. }
13.
14. class _PoliDetailState extends State<PoliDetail> {
15.   Stream<Poli> getData() async* {
16.     Poli data = await PoliService().getById(widget.poli.id.toString());
17.     yield data;
18.   }
19.
20.   @override
21.   Widget build(BuildContext context) {
22.     return Scaffold(
23.       appBar: AppBar(title: Text("Detail Poli")),
24.       body: StreamBuilder(
25.         stream: getData(),
26.         builder: (context, AsyncSnapshot snapshot) {
27.           if (snapshot.hasError) {
28.             return Text(snapshot.error.toString());
29.           }
30.           if (snapshot.connectionState != ConnectionState.done) {
31.             return Center(
32.               child: CircularProgressIndicator(),
33.             );
34.           }
35.           if (!snapshot.hasData &&
36.               snapshot.connectionState == ConnectionState.done) {
37.             return Text('Data Tidak Ditemukan');
38.           }
39.           return Column(
40.             children: [
41.               SizedBox(height: 20),
```

```
42.                Text(
43.                  "Nama Poli : ${snapshot.data.namaPoli}",
44.                  style: TextStyle(fontSize: 20),
45.                ),
46.                SizedBox(height: 20),
47.                Row(
48.                  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
49.                  children: [_tombolUbah(), _tombolHapus()],
50.                )
51.              ],
52.            );
53.          },
54.        ),
55.    );
56.  }
57.
58.  _tombolUbah() {
59.    return StreamBuilder(
60.        stream: getData(),
61.        builder: (context, AsyncSnapshot snapshot) => ElevatedButton(
62.            onPressed: () {
63.              Navigator.push(
64.                  context,
65.                  MaterialPageRoute(
66.                      builder: (context) =>
67.                          PoliUpdateForm(poli: snapshot.data)));
68.            },
69.            style: ElevatedButton.styleFrom(primary: Colors.green),
70.            child: const Text("Ubah")));
71.  }
72.
73.  _tombolHapus() {
74.    return ElevatedButton(
75.        onPressed: () {
76.          AlertDialog alertDialog = AlertDialog(
77.            content: const Text("Yakin ingin menghapus data ini?"),
78.            actions: [
79.              StreamBuilder(
80.                  stream: getData(),
81.                  builder: (context, AsyncSnapshot snapshot) => ElevatedButton(
82.                        onPressed: () async {
83.                          await PoliService()
84.                              .hapus(snapshot.data)
85.                              .then((value) {
86.                            Navigator.pop(context);
87.                            Navigator.pushReplacement(
88.                                context,
89.                                MaterialPageRoute(
90.                                    builder: (context) => PoliPage()));
91.                          });
92.                        },
93.                        child: const Text("YA"),
94.                        style: ElevatedButton.styleFrom(primary: Colors.red),
95.                      )),
96.              ElevatedButton(
97.                onPressed: () {
98.                  Navigator.pop(context);
99.                },
100.                       child: const Text("Tidak"),
101.                       style: ElevatedButton.styleFrom(primary: Colors.green),
102.                     )
103.                   ],
104.                 );
105.                 showDialog(context: context, builder: (context) => alertDialog);
106.               },
107.               style: ElevatedButton.styleFrom(primary: Colors.red),
```

```
                child: const Text("Hapus"));
          }
        }
```

Mengubah Data Poli dari API

Buka file **poli_update_form.dart** pada folder **ui**, kita akan menambahkan sebuah fungsi berikut

```
Future<Poli> getData() async {
    Poli data = await PoliService().getById(widget.poli.id.toString());
    setState(() {
      _namaPoliCtrl.text = data.namaPoli;
    });
    return data;
  }
```

Kemudian pada bagian initState diubah menjadi

```
@override
  void initState() {
    super.initState();
    getData();
  }
```

Kemudian pada bagian _tombolSimpan() diubah menjadi

```
_tombolSimpan() {
    return ElevatedButton(
        onPressed: () async {
          Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
          String id = widget.poli.id.toString();
          await PoliService().ubah(poli, id).then((value) {
            Navigator.pop(context);
            Navigator.pushReplacement(
                context,
                MaterialPageRoute(
                    builder: (context) => PoliDetail(poli: value)));
          });
        },
        child: const Text("Simpan Perubahan"));
  }
```

Adapun keseluruhan kode sebagai berikut

```
1.  import 'package:flutter/material.dart';
2.  import '../model/poli.dart';
3.  import '../service/poli_service.dart';
4.  import 'poli_detail.dart';
5.
6.  class PoliUpdateForm extends StatefulWidget {
7.    final Poli poli;
8.
9.    const PoliUpdateForm({Key? key, required this.poli}) : super(key: key);
10.   _PoliUpdateFormState createState() => _PoliUpdateFormState();
11. }
12.
13. class _PoliUpdateFormState extends State<PoliUpdateForm> {
14.   final _formKey = GlobalKey<FormState>();
15.   final _namaPoliCtrl = TextEditingController();
16.
```

```
17.   Future<Poli> getData() async {
18.     Poli data = await PoliService().getById(widget.poli.id.toString());
19.     setState(() {
20.       _namaPoliCtrl.text = data.namaPoli;
21.     });
22.     return data;
23.   }
24.
25.   @override
26.   void initState() {
27.     super.initState();
28.     getData();
29.   }
30.
31.   @override
32.   Widget build(BuildContext context) {
33.     return Scaffold(
34.       appBar: AppBar(title: const Text("Ubah Poli")),
35.       body: SingleChildScrollView(
36.         child: Form(
37.           key: _formKey,
38.           child: Column(
39.             children: [_fieldNamaPoli(), SizedBox(height: 20), _tombolSimpan()],
40.           ),
41.         ),
42.       ),
43.     );
44.   }
45.
46.   _fieldNamaPoli() {
47.     return TextField(
48.       decoration: const InputDecoration(labelText: "Nama Poli"),
49.       controller: _namaPoliCtrl,
50.     );
51.   }
52.
53.   _tombolSimpan() {
54.     return ElevatedButton(
55.         onPressed: () async {
56.           Poli poli = new Poli(namaPoli: _namaPoliCtrl.text);
57.           String id = widget.poli.id.toString();
58.           await PoliService().ubah(poli, id).then((value) {
59.             Navigator.pop(context);
60.             Navigator.pushReplacement(
61.                 context,
62.                 MaterialPageRoute(
63.                     builder: (context) => PoliDetail(poli: value)));
64.           });
65.         },
66.         child: const Text("Simpan Perubahan"));
67.   }
68.       }
```

TUGAS

Buat pula Detail, Menghapus data dan mengubah data dari API untuk tabel pegawai dan pasien

PERTEMUAN 14

Presentasi Projek

PERTEMUAN 15

Presentasi Projek

PERTEMUAN 16

Ujian Akhir Semester